

Multi-armed Bandits



Thomas Moerland

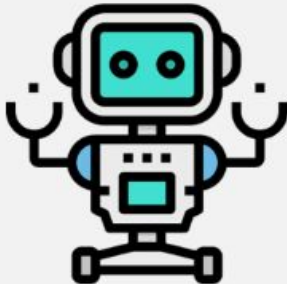
Introduction to Reinforcement Learning
Leiden University

Types of machine learning

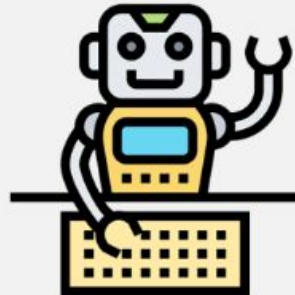


Types of machine learning

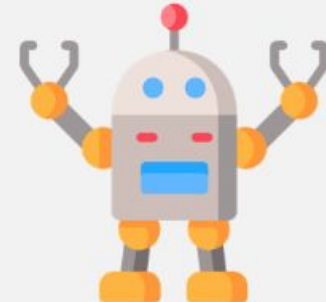
Supervised Learning



Unsupervised Learning

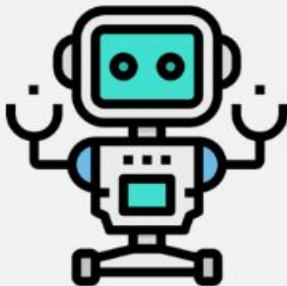


Reinforcement Learning

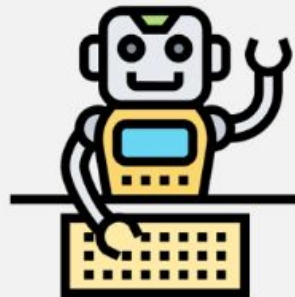


Types of machine learning

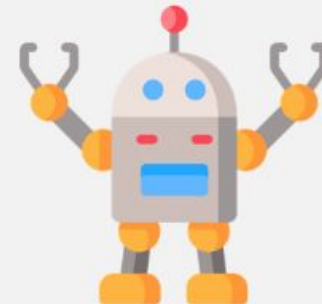
Supervised Learning



Unsupervised Learning



Reinforcement Learning

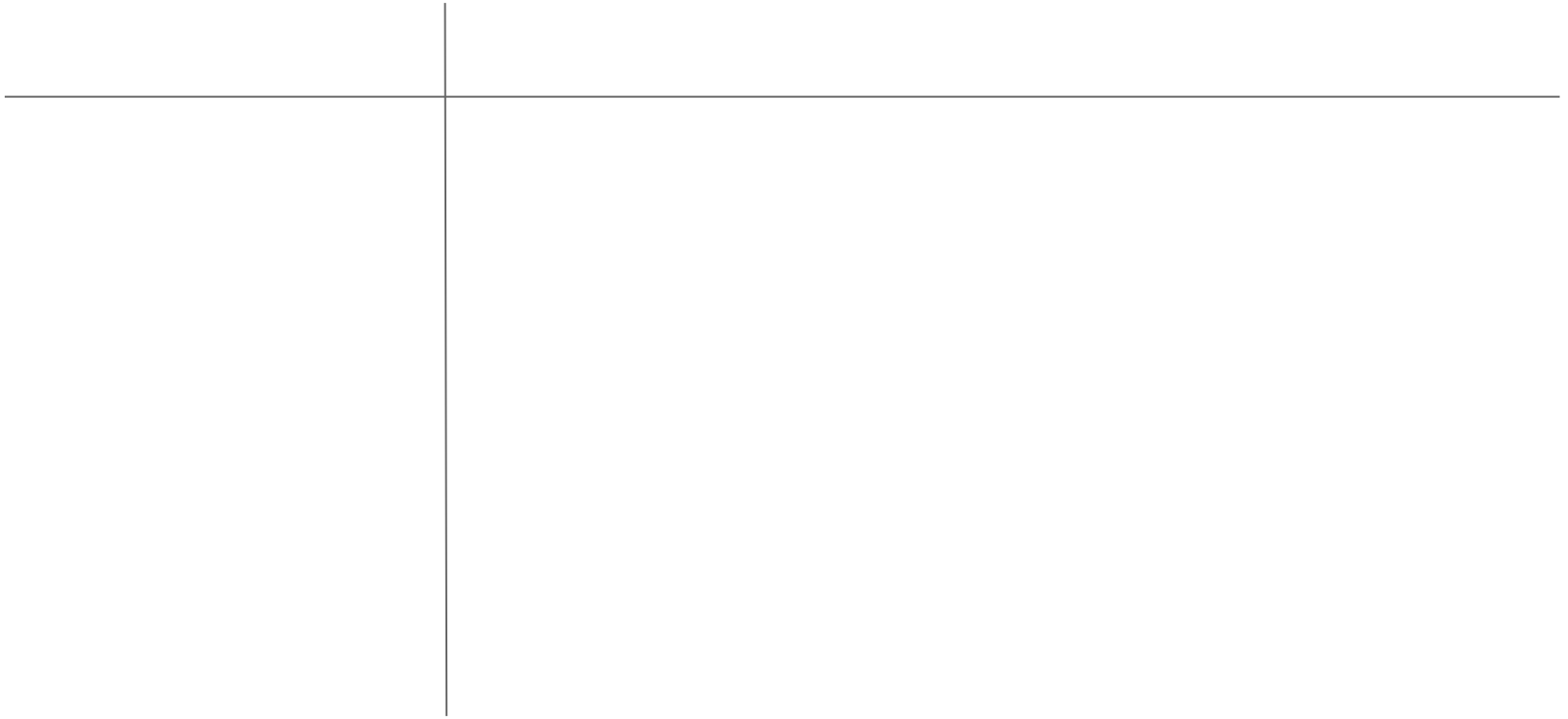


What distinction is really underneath?

Types of machine learning



Types of machine learning



Types of machine learning

Access to data

Dataset given

Actively collect data

Types of machine learning

Access to data

Dataset given

Actively collect data

Information in data

Full feedback

$(x \rightarrow y)$ or $(s \rightarrow a)$

Partial feedback

$(s, a \rightarrow r)$

No feedback

only (x) or (s)

Types of machine learning

	<u>Dataset given</u>	<u>Actively collect data</u>
<u>Full feedback</u>	?	
<u>Partial feedback</u>		
<u>No feedback</u>		

Types of machine learning

	<u>Dataset given</u>	<u>Actively collect data</u>
<u>Full feedback</u>	Supervised learning	
<u>Partial feedback</u>		
<u>No feedback</u>		

Types of machine learning

	<u>Dataset given</u>	<u>Actively collect data</u>
<u>Full feedback</u>	Supervised learning	(Active supervised learning)
<u>Partial feedback</u>		
<u>No feedback</u>		

Types of machine learning

	<u>Dataset given</u>	<u>Actively collect data</u>
<u>Full feedback</u>	Supervised learning	(Active supervised learning)
<u>Partial feedback</u>		Bandits / Reinforcement learning
<u>No feedback</u>		

Types of machine learning

	<u>Dataset given</u>	<u>Actively collect data</u>
<u>Full feedback</u>	Supervised learning	(Active supervised learning)
<u>Partial feedback</u>	(Offline RL)	Bandits / Reinforcement learning
<u>No feedback</u>		

Types of machine learning

	<u>Dataset given</u>	<u>Actively collect data</u>
<u>Full feedback</u>	Supervised learning	(Active supervised learning)
<u>Partial feedback</u>	(Offline RL)	Bandits / Reinforcement learning
<u>No feedback</u>	Unsupervised learning	

Types of machine learning

	<u>Dataset given</u>	<u>Actively collect data</u>
<u>Full feedback</u>	Supervised learning	(Active supervised learning)
<u>Partial feedback</u>	(Offline RL)	Bandits / Reinforcement learning
<u>No feedback</u>	Unsupervised learning	(Active unsupervised learning)

Types of machine learning

	<u>Dataset given</u>	<u>Actively collect data</u>
<u>Full feedback</u>	Supervised learning	(Active supervised learning)
<u>Partial feedback</u>	(Offline RL)	Bandits / Reinforcement learning
<u>No feedback</u>	Unsupervised learning	(Active unsupervised learning)

Bandits versus reinforcement learning



Bandits versus reinforcement learning

Bandit = 1-step decision-making problem

Bandits versus reinforcement learning

Bandit	=	1-step decision-making problem
Reinforcement learning	=	sequential decision-making problem

Bandits versus reinforcement learning

Bandit	=	1-step decision-making problem
Reinforcement learning	=	sequential decision-making problem

	Active data collection	Partial feedback	Multi-step
Bandit			
RL			

Bandits versus reinforcement learning

Bandit = 1-step decision-making problem
Reinforcement learning = sequential decision-making problem

	Active data collection	Partial feedback	Multi-step
Bandit	+	+	-
RL			

Bandits versus reinforcement learning

Bandit = 1-step decision-making problem
Reinforcement learning = sequential decision-making problem

	Active data collection	Partial feedback	Multi-step
Bandit	+	+	-
RL	+	+	+

Bandits versus reinforcement learning

Bandit = 1-step decision-making problem
Reinforcement learning = sequential decision-making problem

	Active data collection	Partial feedback	Multi-step
Bandit	+	+	-
RL	+	+	+



Bandits illustrate the
exploration/exploitation
problem

Bandits versus reinforcement learning

Bandit = 1-step decision-making problem
Reinforcement learning = sequential decision-making problem

	Active data collection	Partial feedback	Multi-step
Bandit	+	+	-
RL	+	+	+

Bandits illustrate the exploration/exploitation problem

Without the temporal credit assignment problem (yet)

Bandits versus reinforcement learning

Bandit = 1-step decision-making problem
Reinforcement learning = sequential decision-making problem

	Active data collection	Partial feedback	Multi-step
Bandit	+	+	-
RL	+	+	+

Bandits are a great stepping stone into RL

Bandits illustrate the exploration/exploitation problem

Without the temporal credit assignment problem (yet)

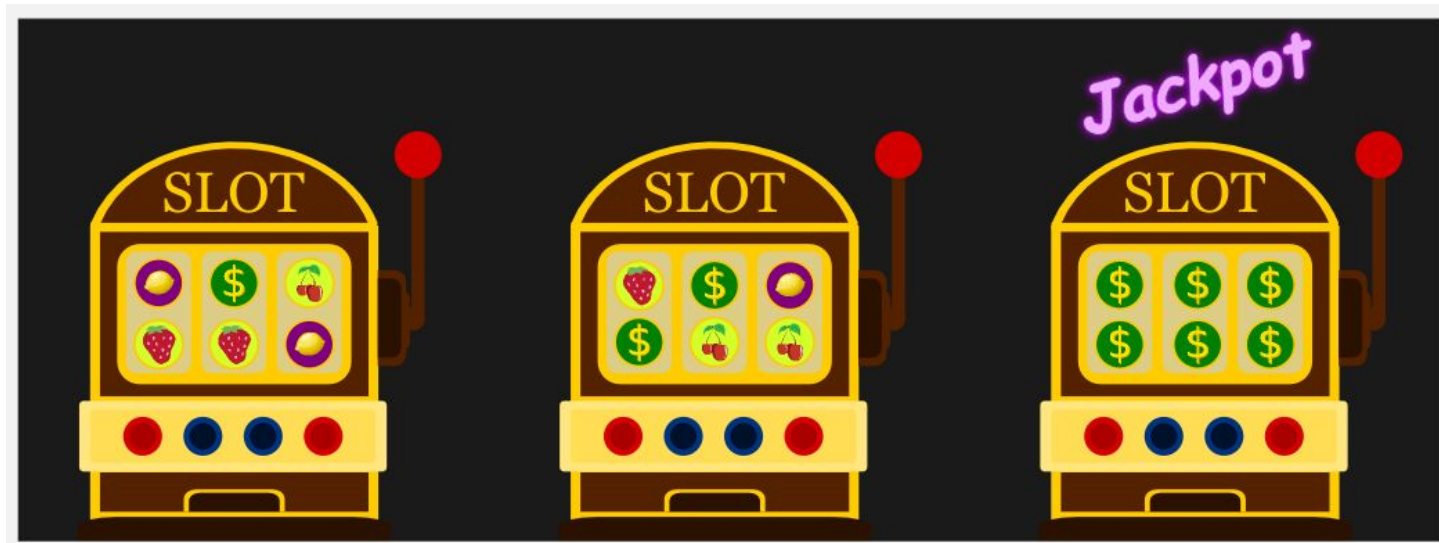
Content

1. Bandit definition
2. Exploration/exploitation
3. Updating a mean
4. Bandit algorithms
 - a. Random perturbation
 - b. Optimistic initialization
 - c. Optimism in the face of uncertainty
5. Contextual bandits & MDPs

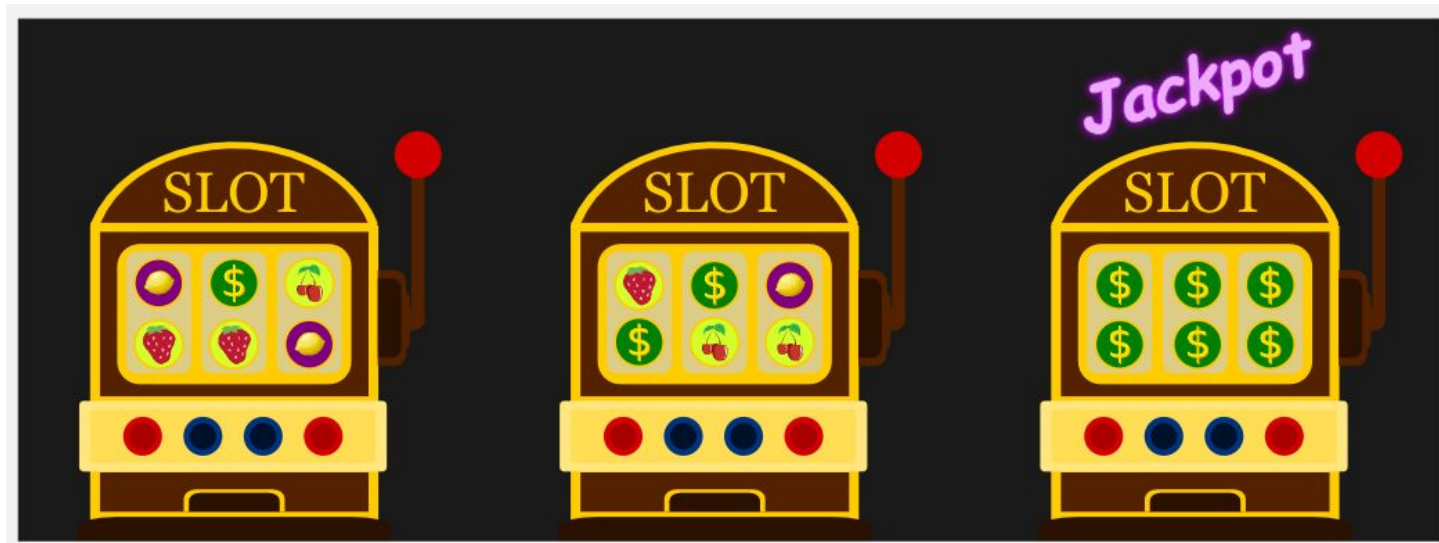
Part 1:

Bandit definition

Multi-armed bandit

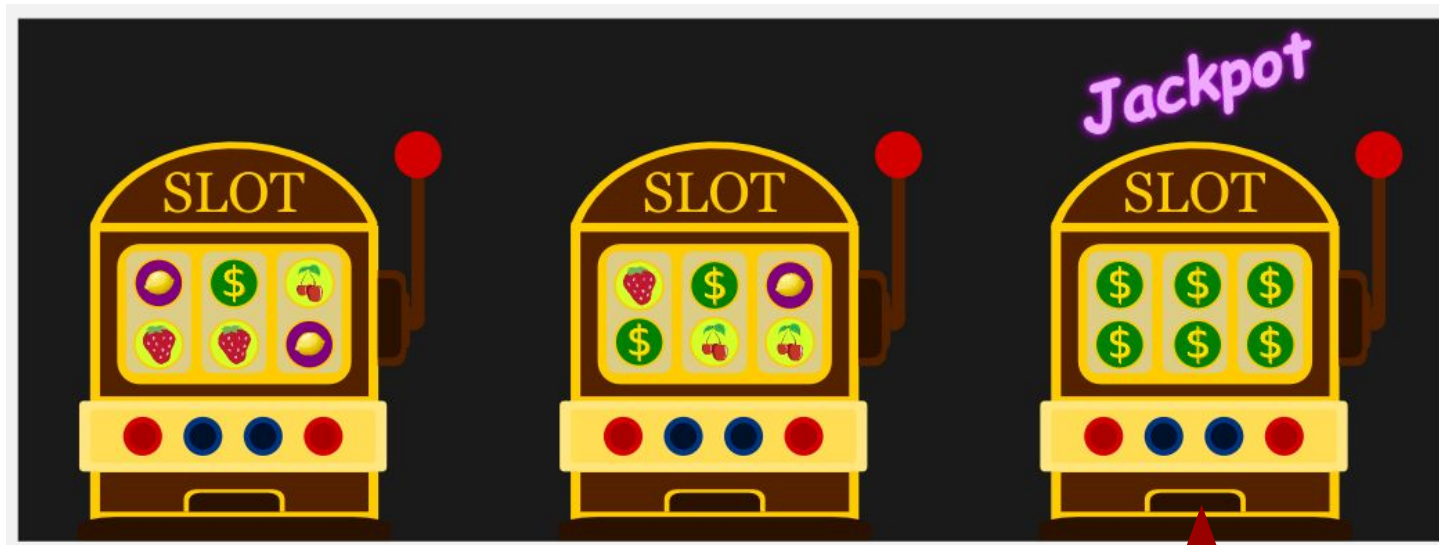


Multi-armed bandit



Don't know the pay-off of each arm/action

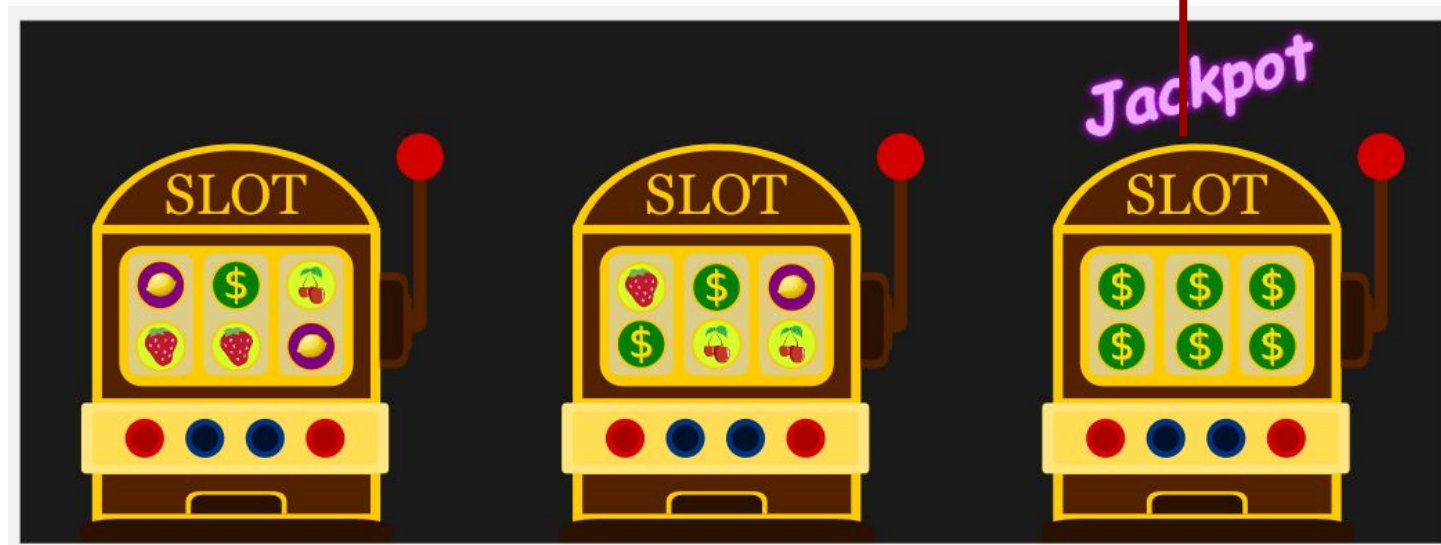
Multi-armed bandit



Let's try an action
(`pull an arm')

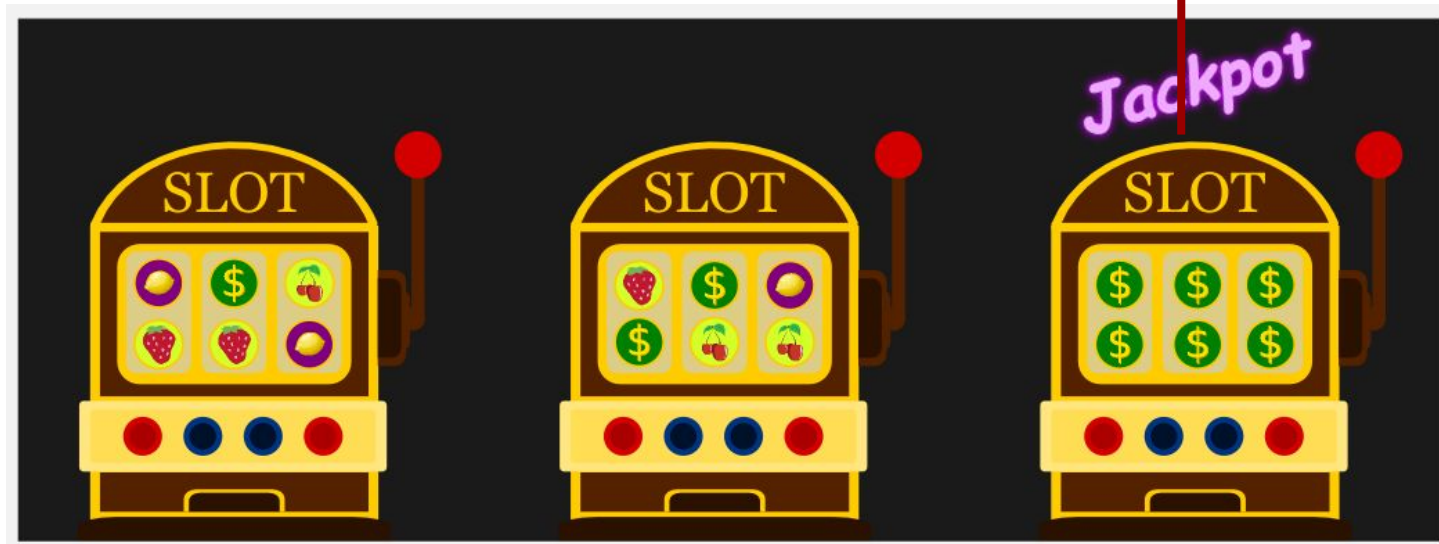
Multi-armed bandit

Reward = -1

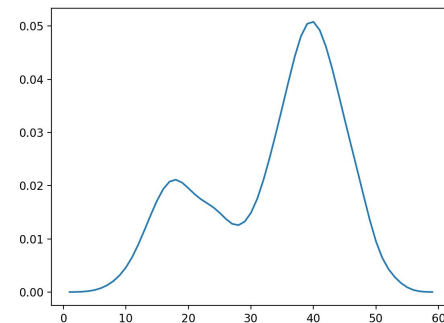


Multi-armed bandit

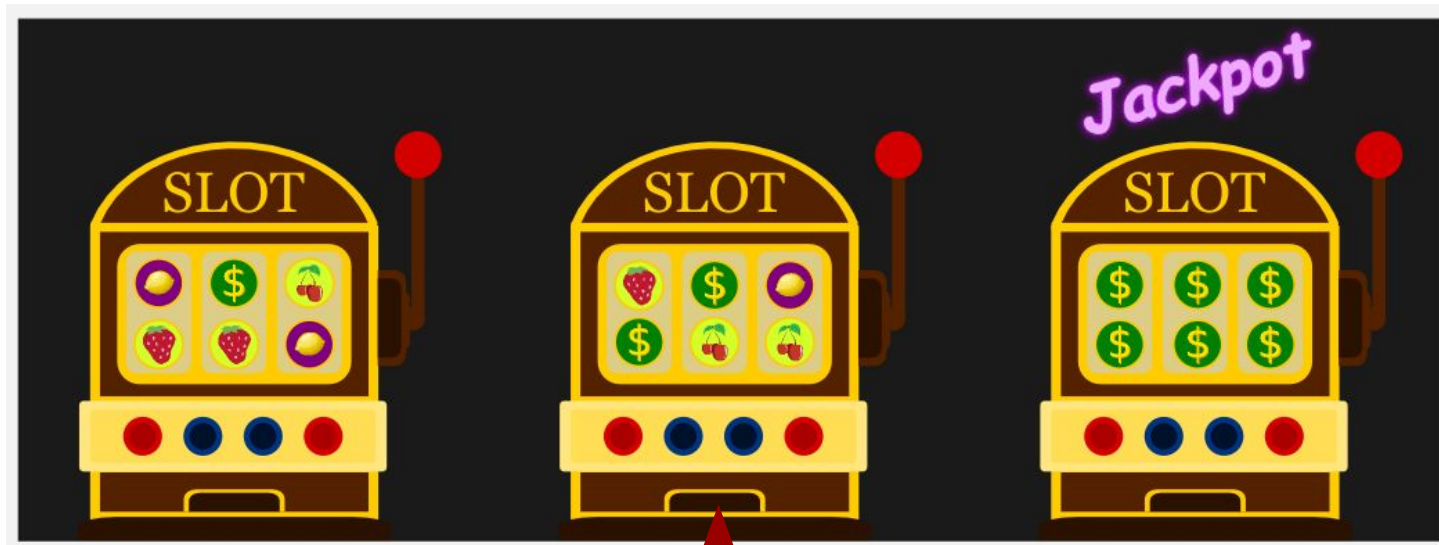
Reward = -1



(noisy sample from unknown
reward distribution)



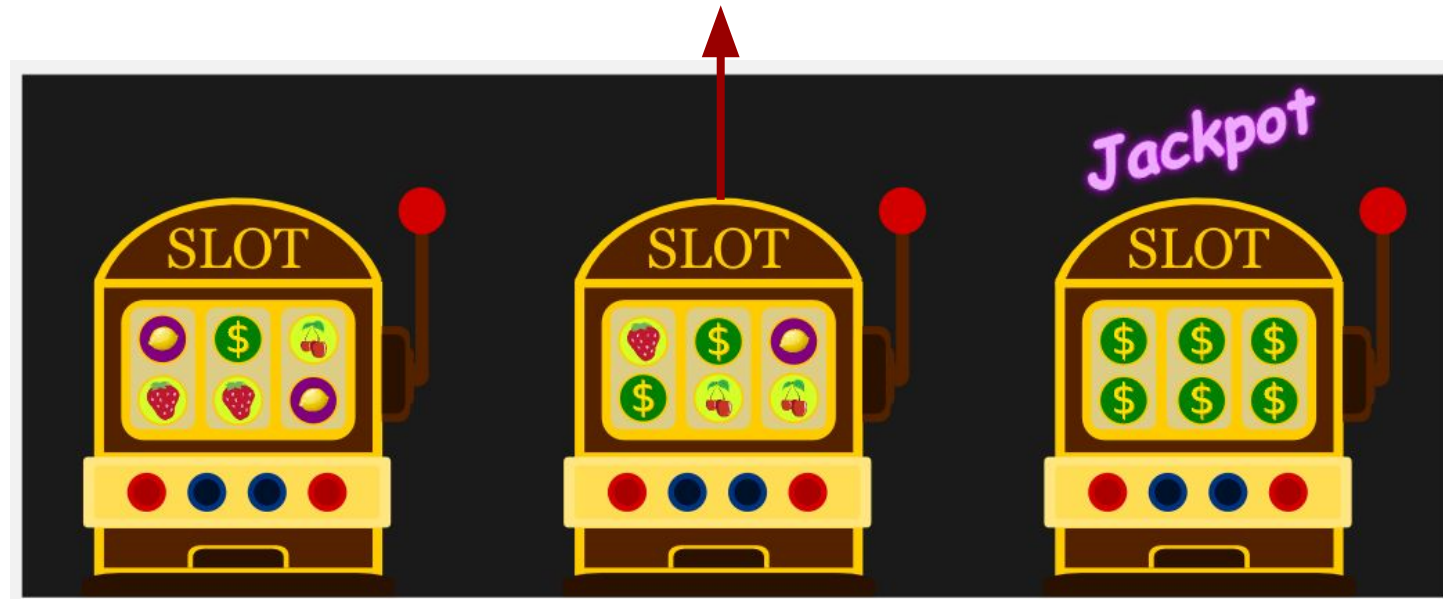
Multi-armed bandit



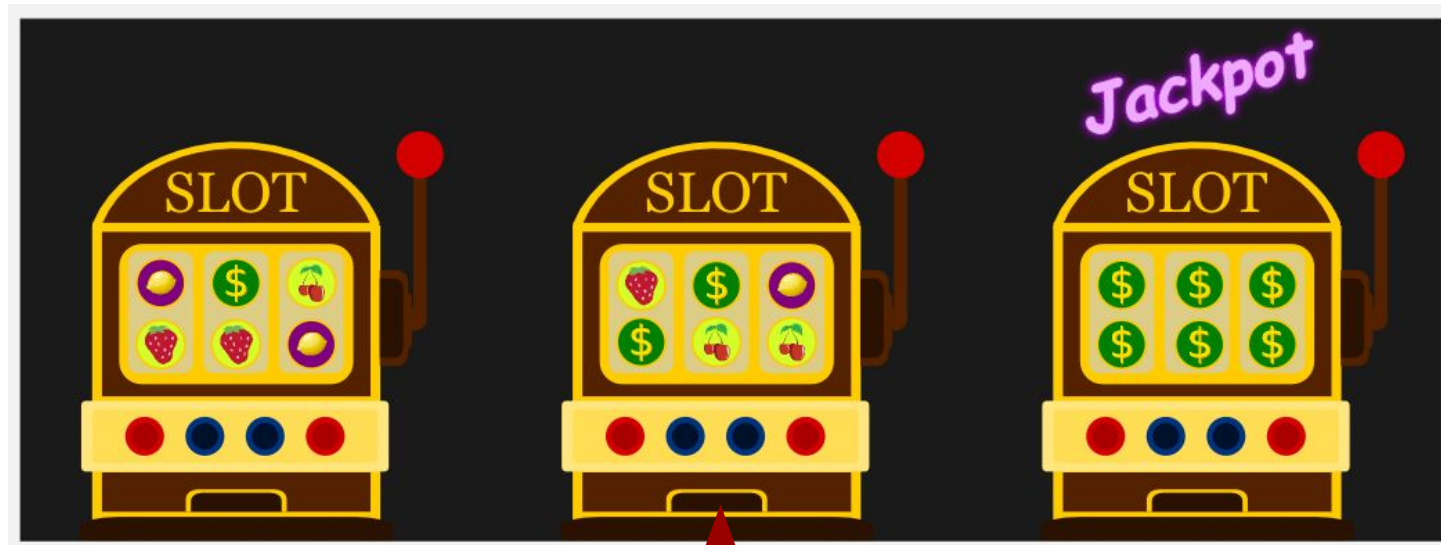
Let's try another one

Multi-armed bandit

Reward = 4



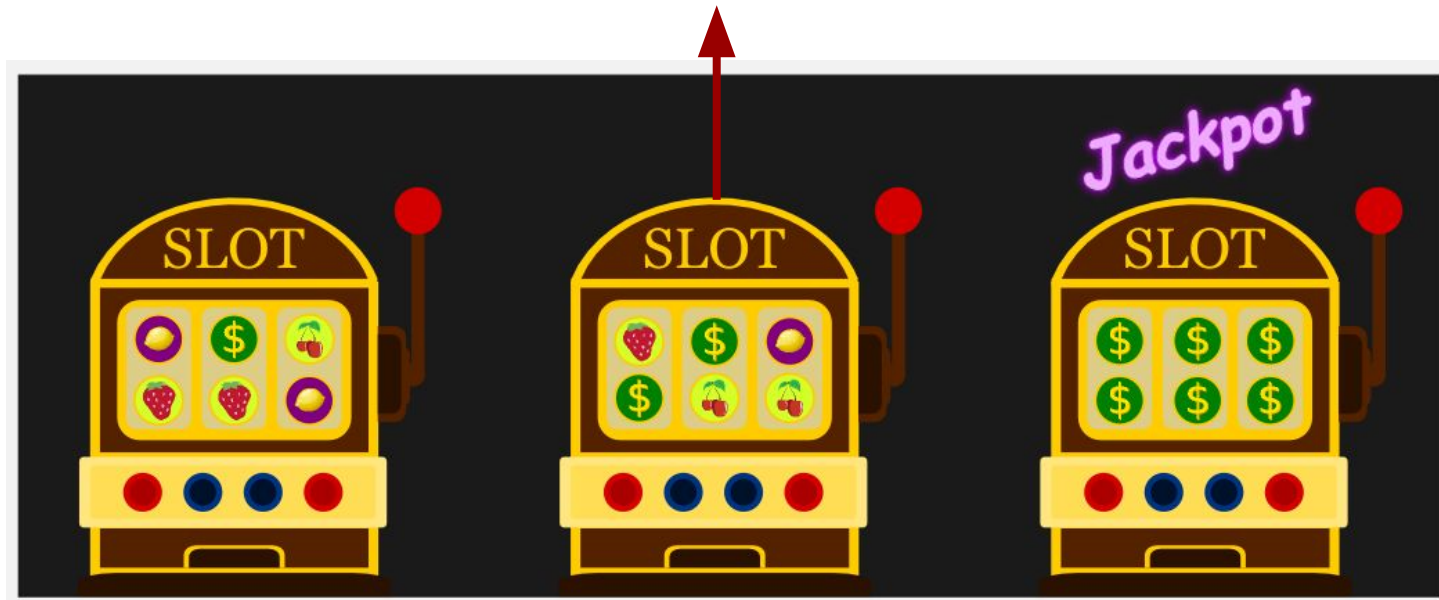
Multi-armed bandit



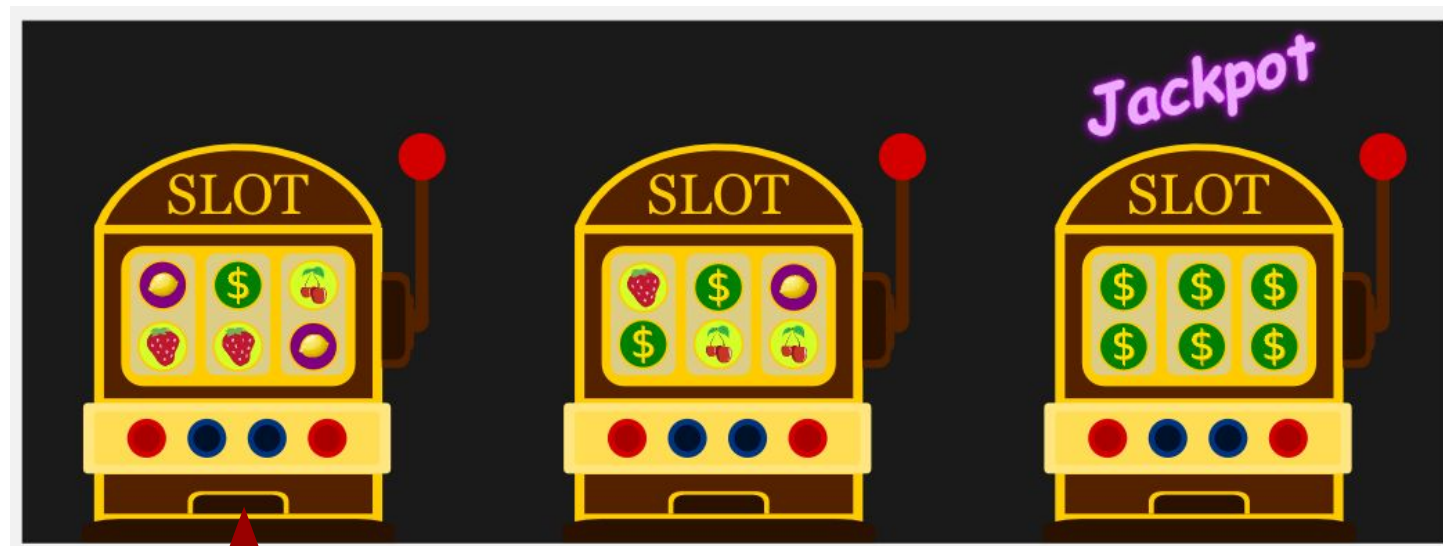
Let's try it again

Multi-armed bandit

Reward = -3

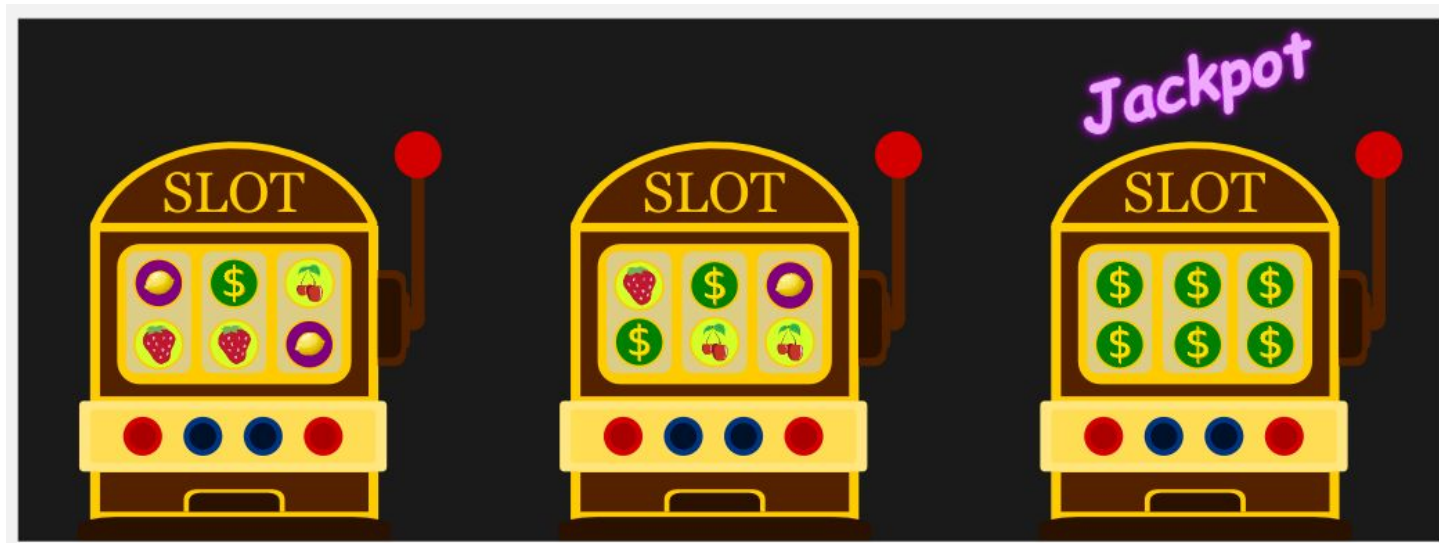


Multi-armed bandit



etc.

Multi-armed bandit

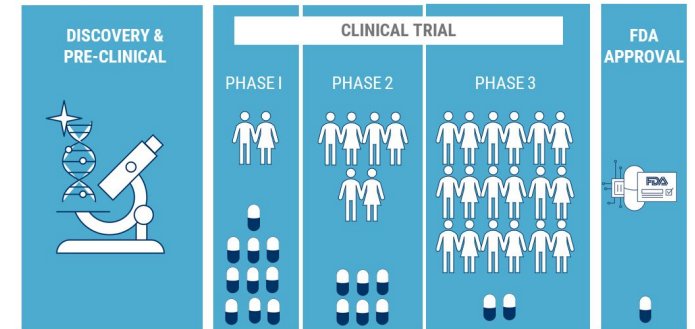


Our aim: maximize sum of all rewards

Applications

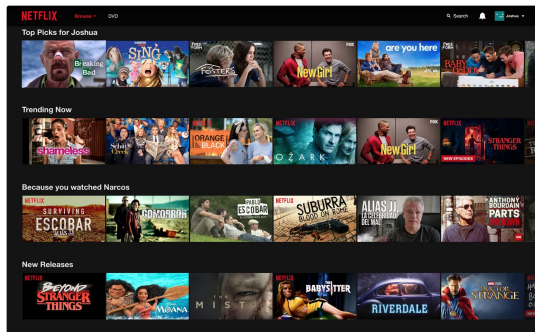


Applications

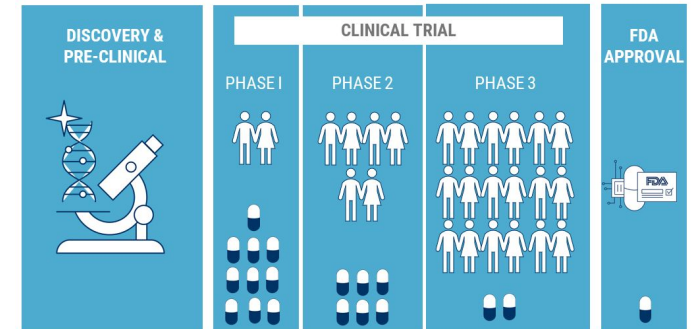


Medicine:
clinical trials

Applications

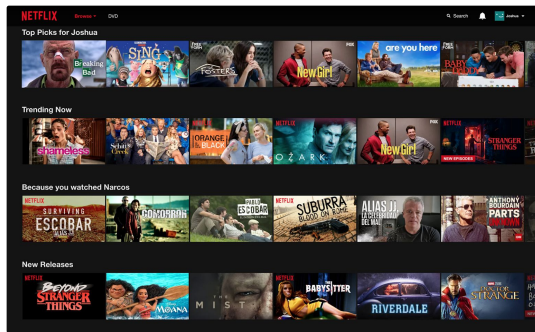


**Advertisement:
recommender systems**



**Medicine:
clinical trials**

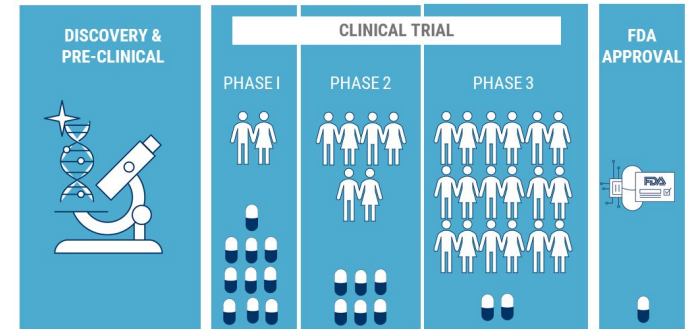
Applications



Advertisement:
recommender systems

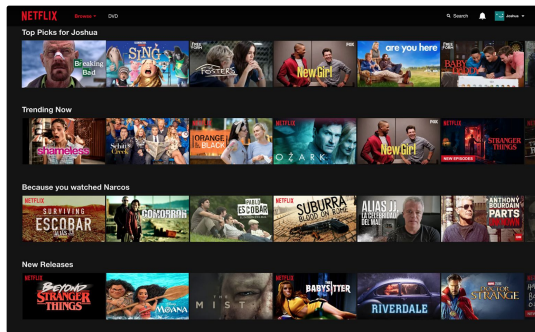


Finance:
portfolio management



Medicine:
clinical trials

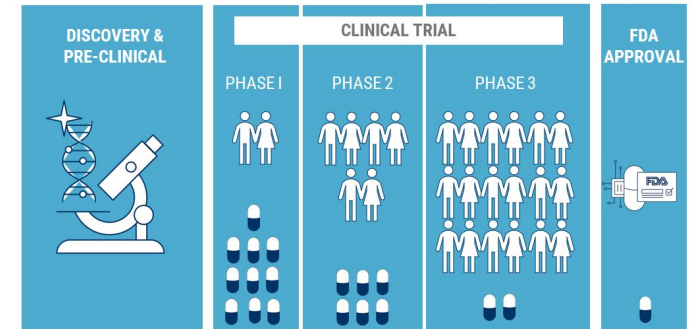
Applications



Advertisement:
recommender systems



Finance:
portfolio management



Medicine:
clinical trials

Also: Important
building block for
reinforcement
learning

Multi-armed bandit: definition



Multi-armed bandit: definition

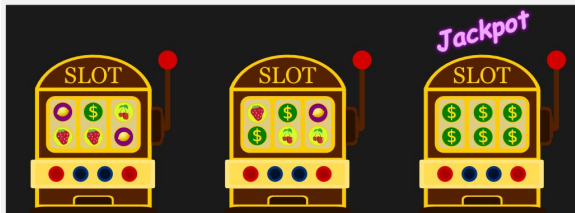
$$\langle \mathcal{A}, p(r|a) \rangle$$

Multi-armed bandit: definition

$$\langle \mathcal{A}, p(r|a) \rangle$$



Set of actions
e.g.: $\{1,2,3\}$

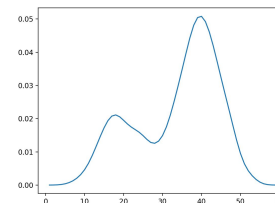
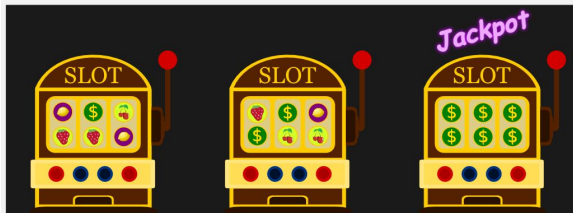


Multi-armed bandit: definition

$$\langle \mathcal{A}, p(r|a) \rangle$$

Set of actions
e.g.: {1,2,3}

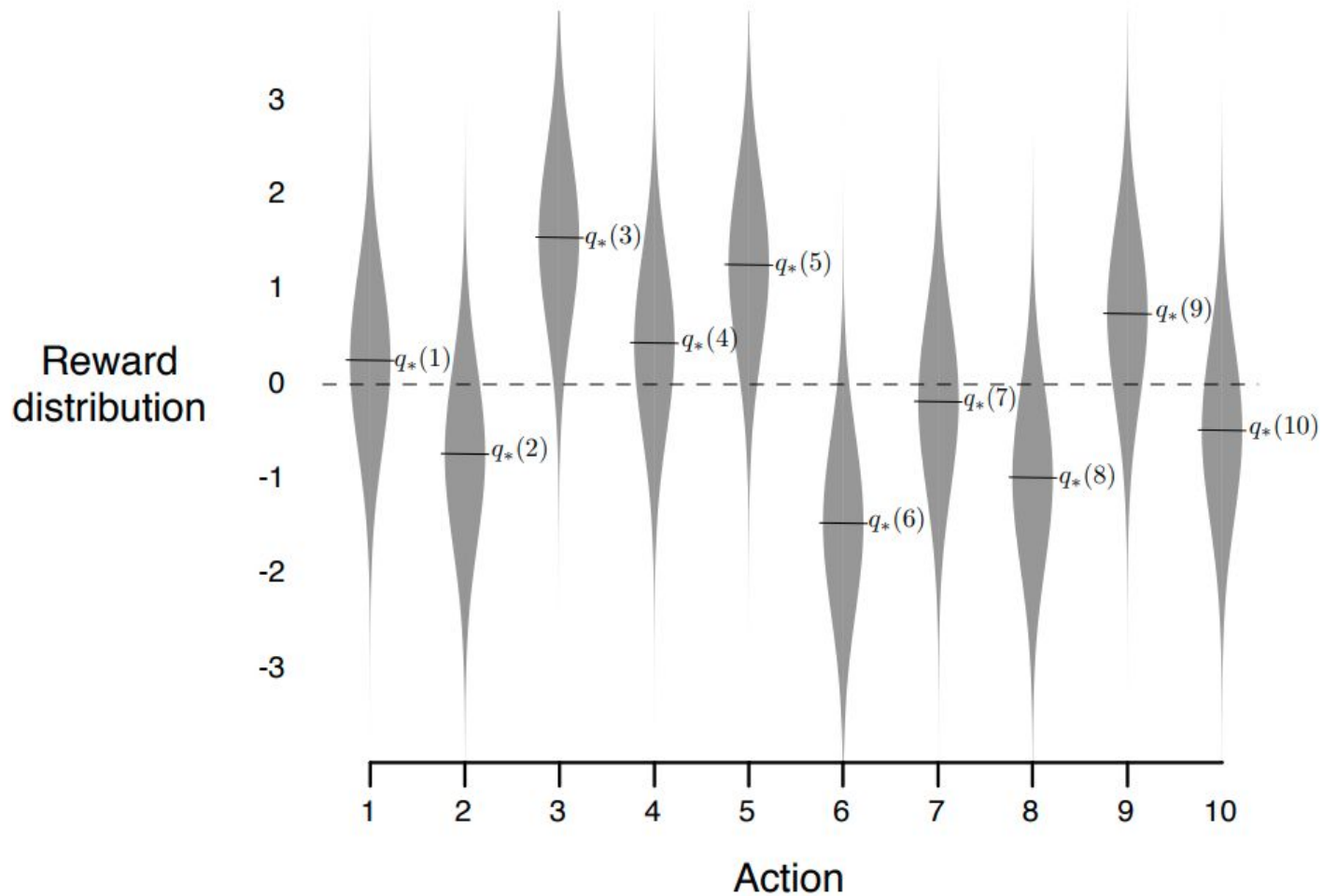
Distribution of rewards
for each action



Multi-armed bandit: example



Multi-armed bandit: example



Action value $Q(a)$

- Every arm has a *mean pay-off* $Q(a)$:

$$Q(a) = \mathbb{E}_{r \sim p(r|a)}[r]$$

Action value $Q(a)$

- Every arm has a *mean pay-off* $Q(a)$:

$$Q(a) = \mathbb{E}_{r \sim p(r|a)}[r]$$



“expectation”

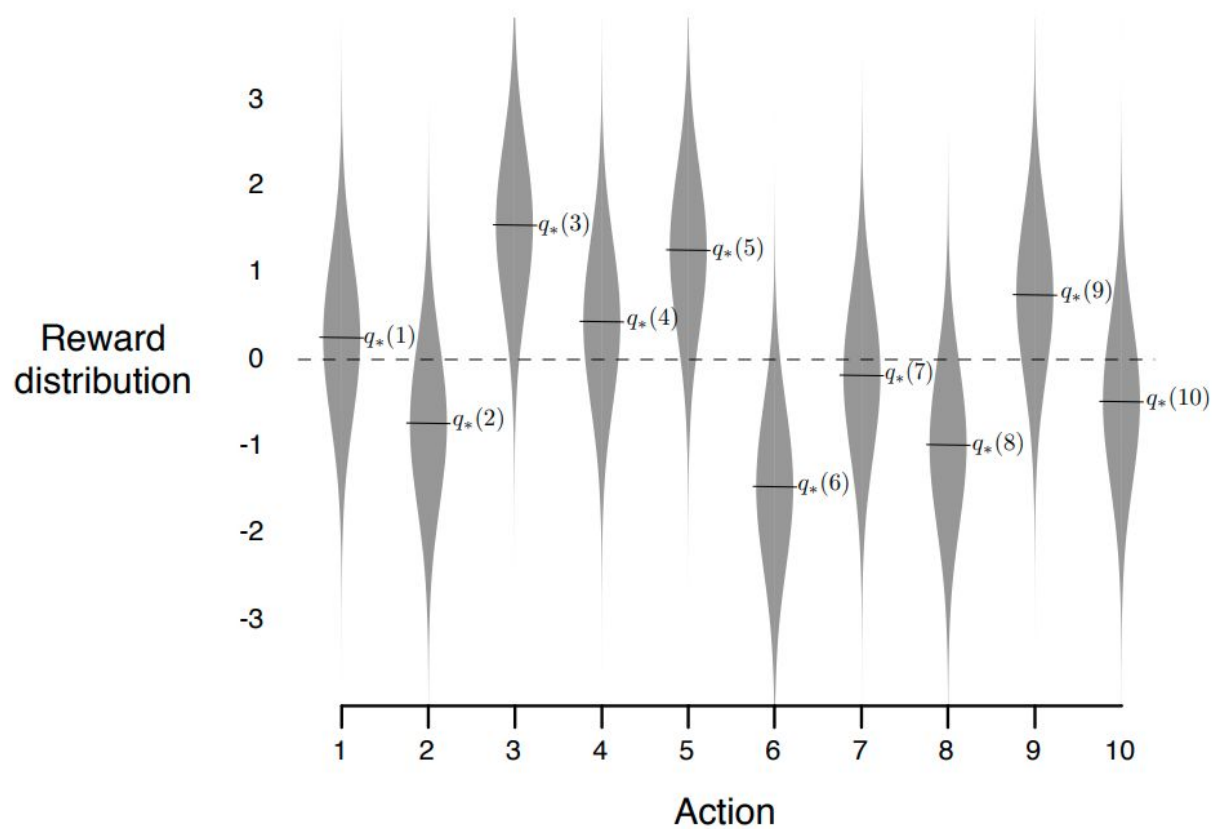
Action value $Q(a)$

- Every arm has a *mean pay-off* $Q(a)$:

$$Q(a) = \mathbb{E}_{r \sim p(r|a)}[r]$$

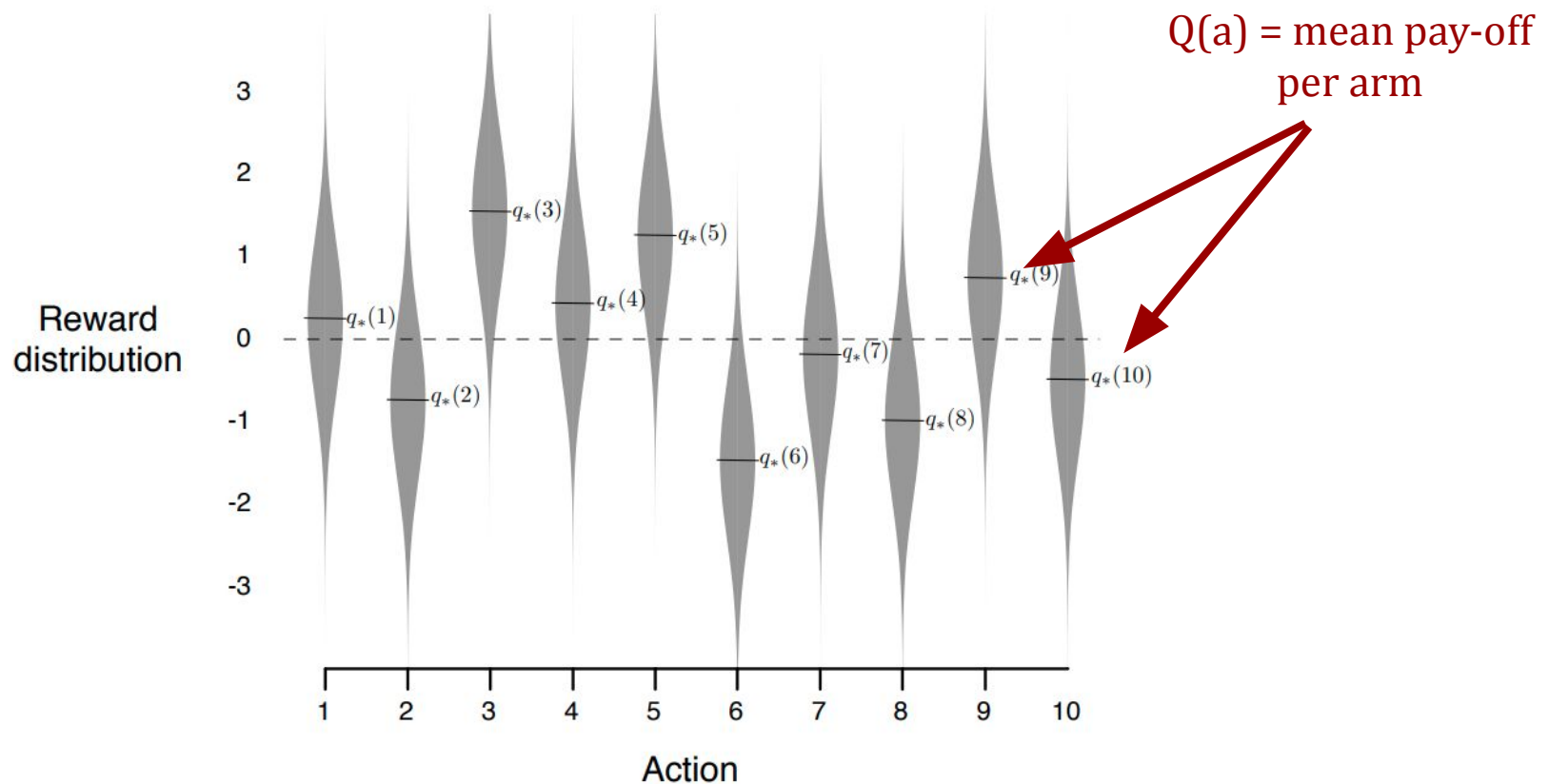
We call $Q(a)$ the “**action value**”

Action value $Q(a)$



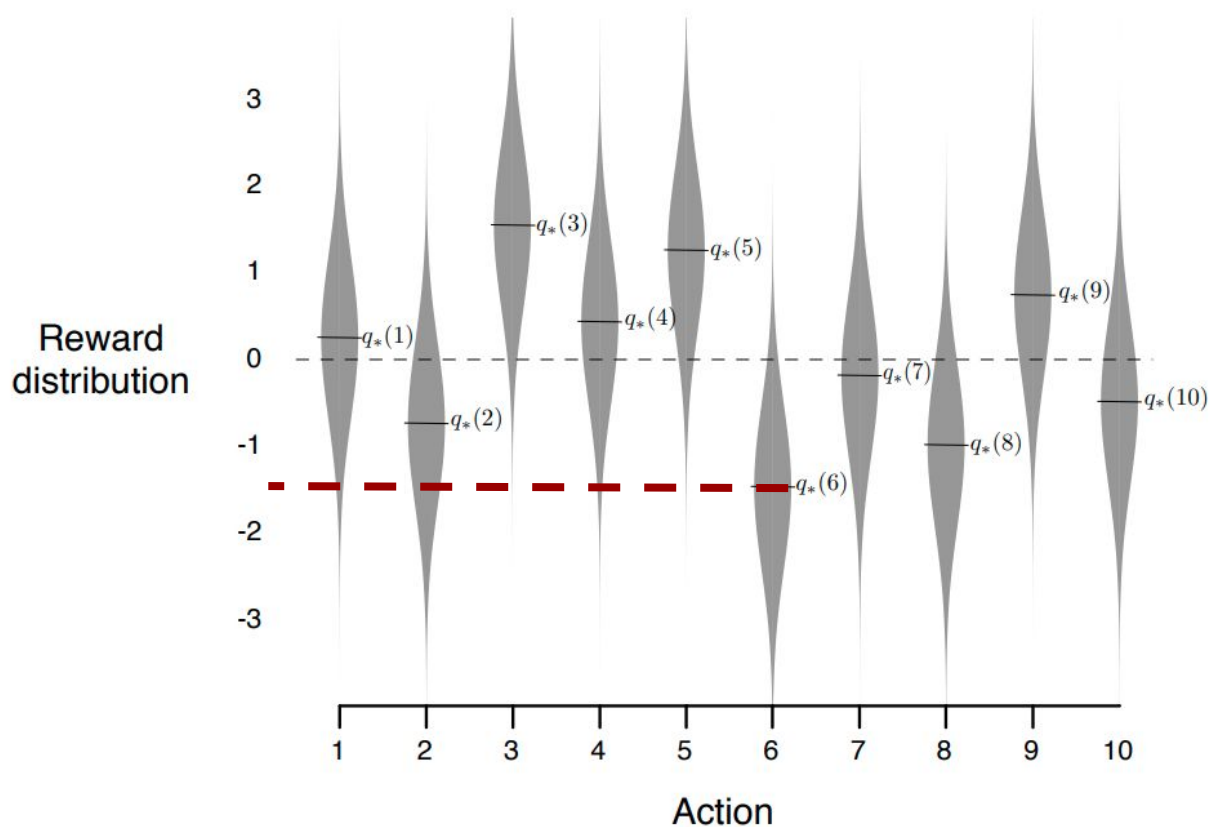
Question: what do you estimate $Q(a=6)$?

Action value $Q(a)$



Question: what do you estimate $Q(a=6)$?

Action value $Q(a)$



Question: what do you estimate $Q(a=6)$?

Answer: $Q(a=6) \approx -1.5$

Action selection



Action selection

$$\pi(a)$$

“Policy”

=

Probability distribution over the (discrete) action space

Action selection

$$\pi(a)$$

“Policy”

=

Probability distribution over the (discrete) action space

Example:

$\pi(a = 1)$	$\pi(a = 2)$	$\pi(a = 3)$	$\pi(a = 4)$
0.2	0.7	0.0	0.1

Action selection

$$\pi(a)$$

Policy can also be **implicitly** stored

Action selection

$$\pi(a)$$

Policy can also be **implicitly** stored

Insight: close relation between action value $Q(a)$ and policy $\pi(a)$ (higher action value should get higher policy probability)

Action selection

$$\pi(a)$$

Policy can also be **implicitly** stored

Insight: close relation between action value $Q(a)$ and policy $\pi(a)$ (higher action value should get higher policy probability)

1. Store the action value estimates:

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

Action selection

$$\pi(a)$$

Policy can also be **implicitly** stored

Insight: close relation between action value $Q(a)$ and policy $\pi(a)$ (higher action value should get higher policy probability)

1. Store the action value estimates:

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

2. Make policy a function of the action values:

$$\pi = f(Q(a))$$

Action selection

$$\pi(a)$$

Policy can also be **implicitly** stored

Insight: close relation between action value $Q(a)$ and policy $\pi(a)$ (higher action value should get higher policy probability)

1. Store the action value estimates:

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

2. Make policy a function of the action values:

$$\pi = f(Q(a))$$

Most common: will see different implicit policies (forms of f)!

Objective

What do we actually want to achieve in the bandit setting?

Objective

What do we actually want to achieve in the bandit setting?

Repeatedly choose the right action and get as much reward as possible!

Objective

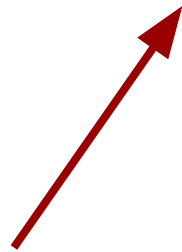
What do we actually want to achieve in the bandit setting?

$$J_T(\pi) = \mathbb{E}_{a_t \sim \pi(a), r_t \sim p(r|a_t)} \left[\sum_{t=1}^T r_t \right]$$

Objective

What do we actually want to achieve in the bandit setting?

$$J_T(\pi) = \mathbb{E}_{a_t \sim \pi(a), r_t \sim p(r|a_t)} \left[\sum_{t=1}^T r_t \right]$$



Objective is a function of our policy

Objective

What do we actually want to achieve in the bandit setting?

$$J_T(\pi) = \mathbb{E}_{a_t \sim \pi(a), r_t \sim p(r|a_t)} \left[\sum_{t=1}^T r_t \right]$$

Objective is a function of our policy

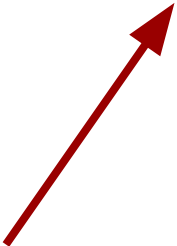
Average performance over our
policy and the stochastic rewards

Objective

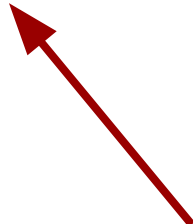
What do we actually want to achieve in the bandit setting?

$$J_T(\pi) = \mathbb{E}_{a_t \sim \pi(a), r_t \sim p(r|a_t)} \left[\sum_{t=1}^T r_t \right]$$

Objective is a function of our policy



Average performance over our policy and the stochastic rewards



Sum of rewards over T decisions



Objective

What do we actually want to achieve in the bandit setting?

$$J_T(\pi) = \mathbb{E}_{a_t \sim \pi(a), r_t \sim p(r|a_t)} \left[\sum_{t=1}^T r_t \right]$$

But our goal is to find the *best* policy

Objective

What do we actually want to achieve in the bandit setting?

$$J_T(\pi) = \mathbb{E}_{a_t \sim \pi(a), r_t \sim p(r|a_t)} \left[\sum_{t=1}^T r_t \right]$$

$$\pi^* = \arg \max_{\pi} J_T(\pi)$$

Objective

What do we actually want to achieve in the bandit setting?

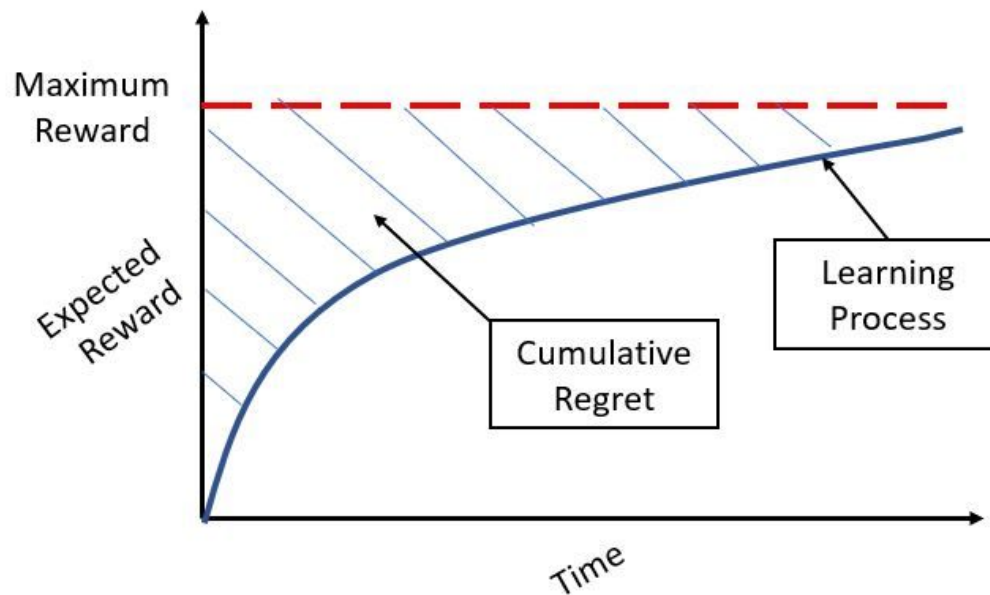
$$J_T(\pi) = \mathbb{E}_{a_t \sim \pi(a), r_t \sim p(r|a_t)} \left[\sum_{t=1}^T r_t \right]$$

$$\pi^* = \arg \max_{\pi} J_T(\pi)$$

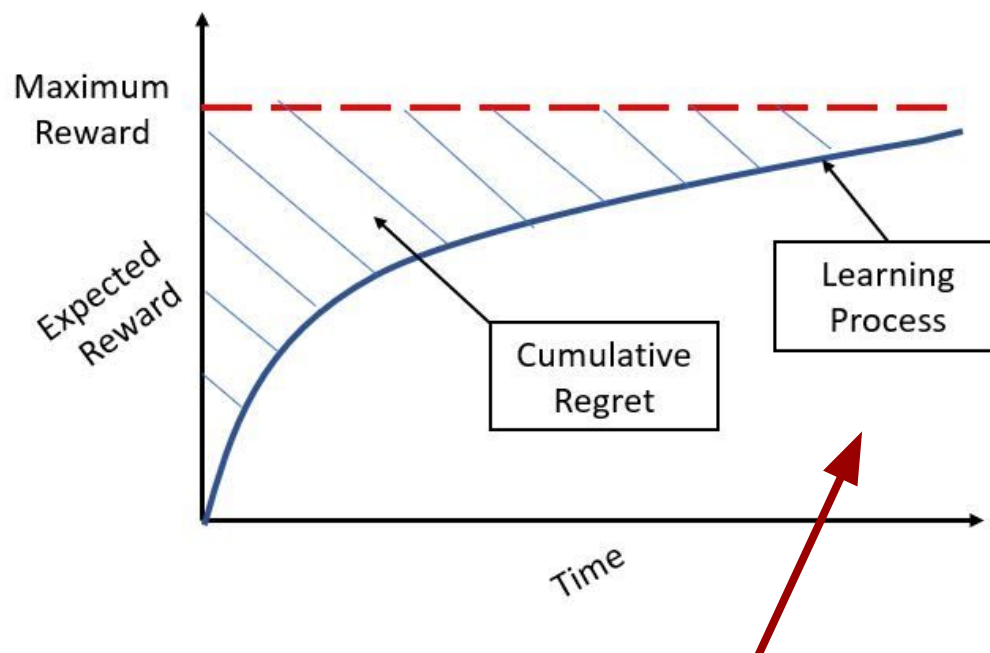
We want to find/specify π^* , the policy that maximizes our average pay-off!

Objective: visual interpretation

Objective: visual interpretation



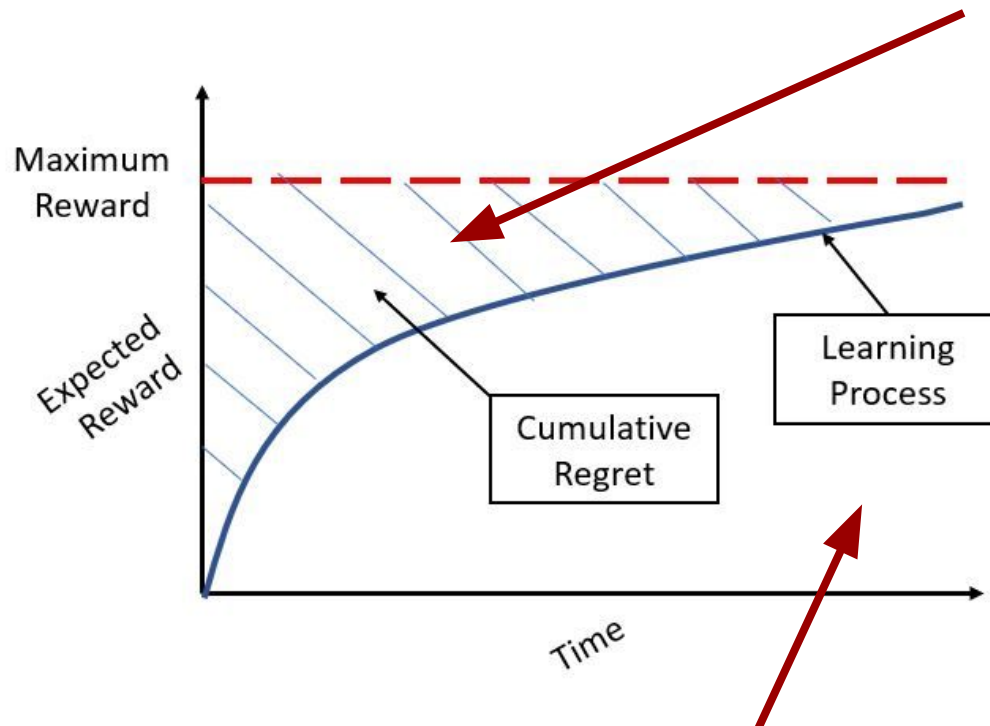
Objective: visual interpretation



The objective from the previous slide maximizes the area below the learning curve

Objective: visual interpretation

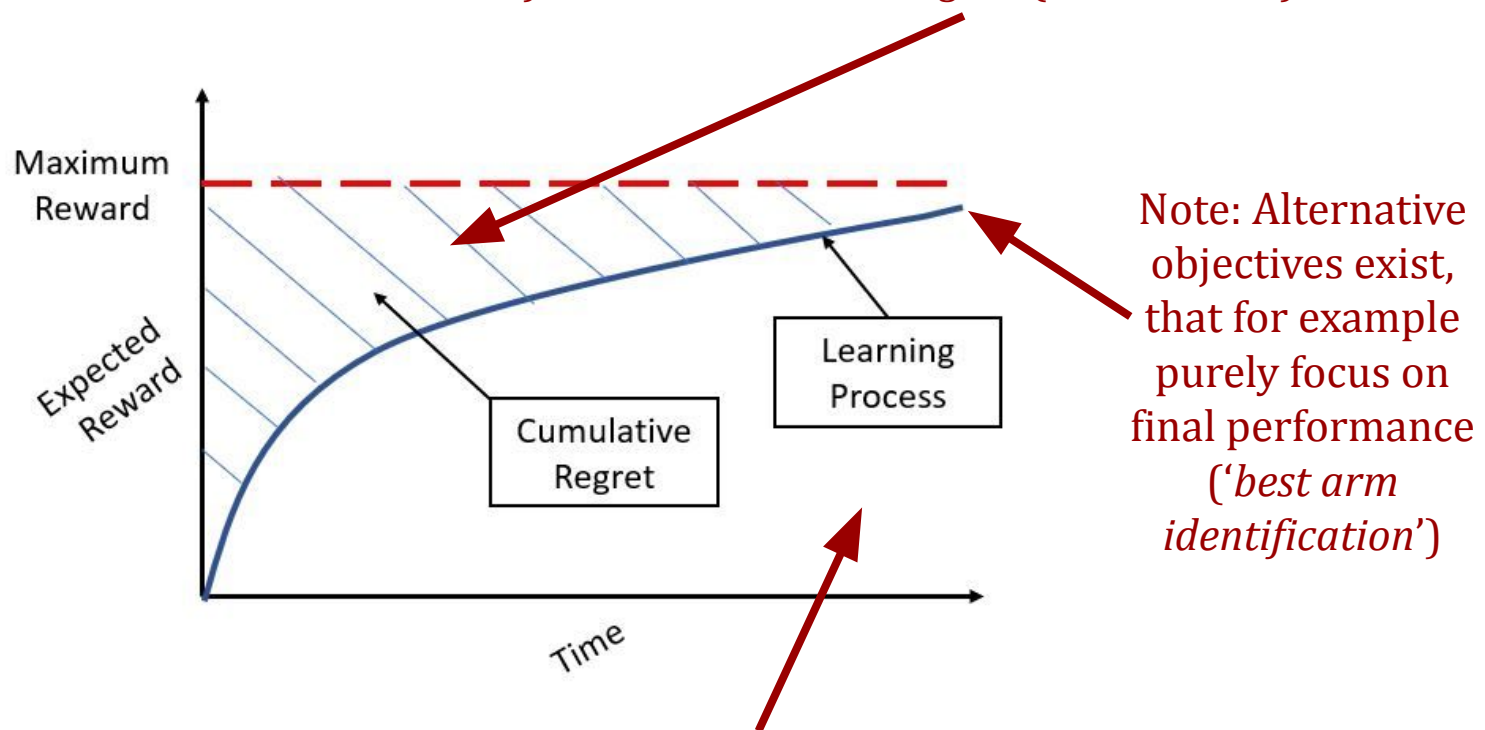
Which is equivalent to minimizing the area above the curve (with respect to the mean reward of the best action), which we call the *regret* (shaded area)



The objective from the previous slide maximizes the area below the learning curve

Objective: visual interpretation

Which is equivalent to minimizing the area above the curve (with respect to the mean reward of the best action), which we call the *regret* (shaded area)

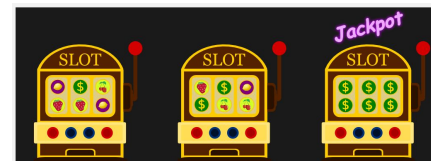


The objective from the previous slide maximizes the area below the learning curve

Summary

- *Bandit definition:*

$$\langle \mathcal{A}, p(r|a) \rangle$$



- *Action value:*

$$Q(a) = \mathbb{E}_{r \sim p(r|a)}[r]$$

$$\frac{Q(a=1)}{1.2} \quad \frac{Q(a=2)}{0.3} \quad \frac{Q(a=3)}{-2.4} \quad \frac{Q(a=4)}{3.5}$$

- *Policy:*

$$\pi(a)$$

$$\frac{\pi(a=1)}{0.2} \quad \frac{\pi(a=2)}{0.7} \quad \frac{\pi(a=3)}{0.0} \quad \frac{\pi(a=4)}{0.1}$$

- *Objective:*

$$J_T(\pi) = \mathbb{E}_{a_t \sim \pi(a), r_t \sim p(r|a_t)} \left[\sum_{t=1}^T r_t \right]$$

Part 2:

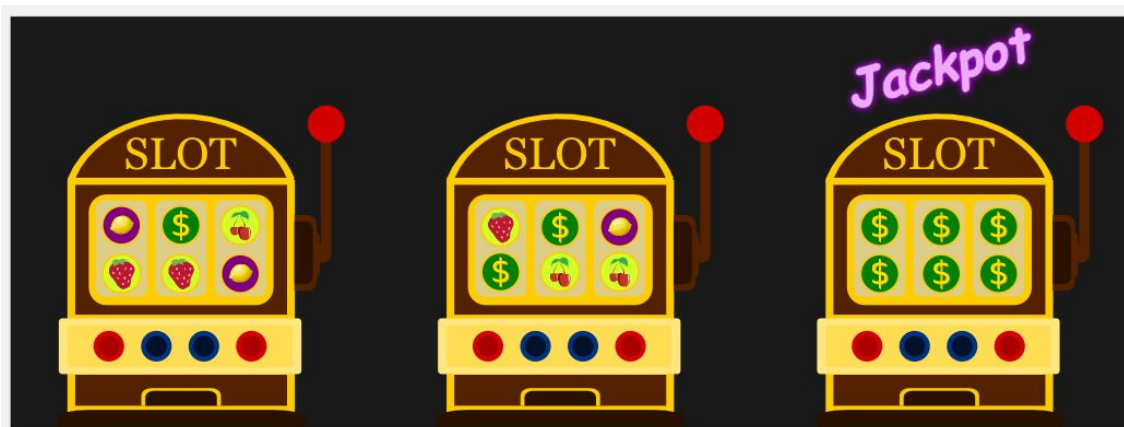
Exploration/Exploitation

Exploration/exploitation

Why is it challenging to find a good policy?

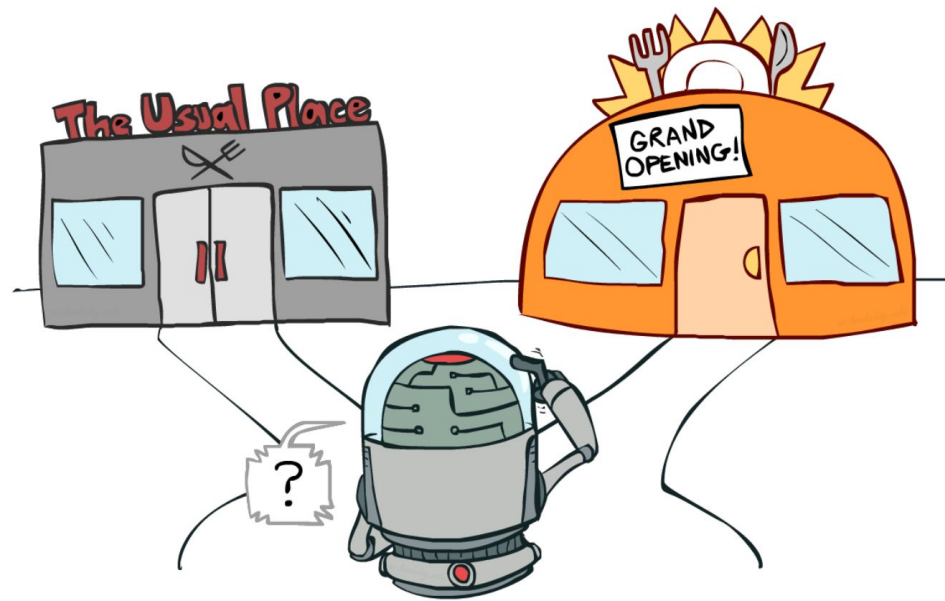
Exploration/exploitation

Why is it challenging to find a good policy?



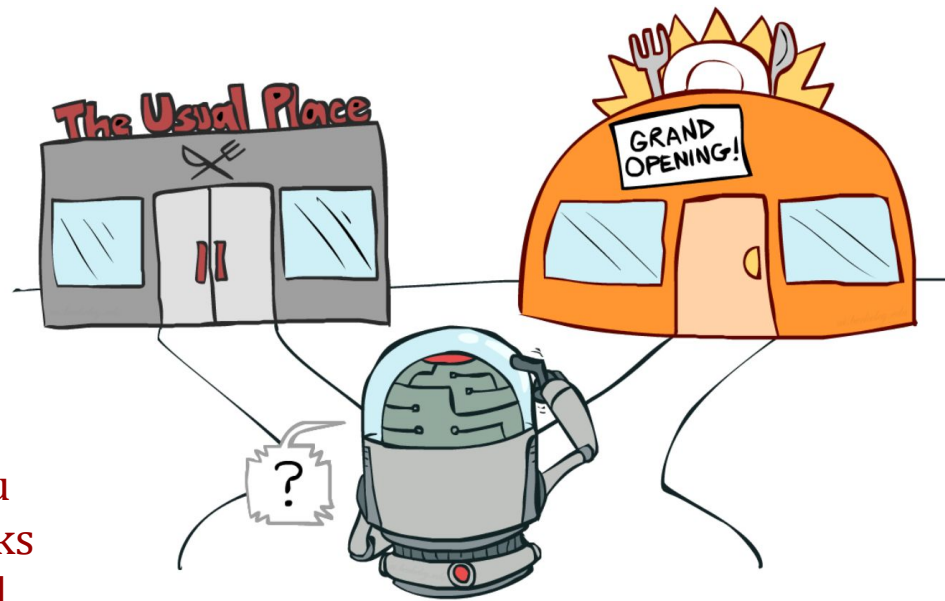
Question: You can play 100 rounds at these slotmachines. How would you act?

Exploration/exploitation

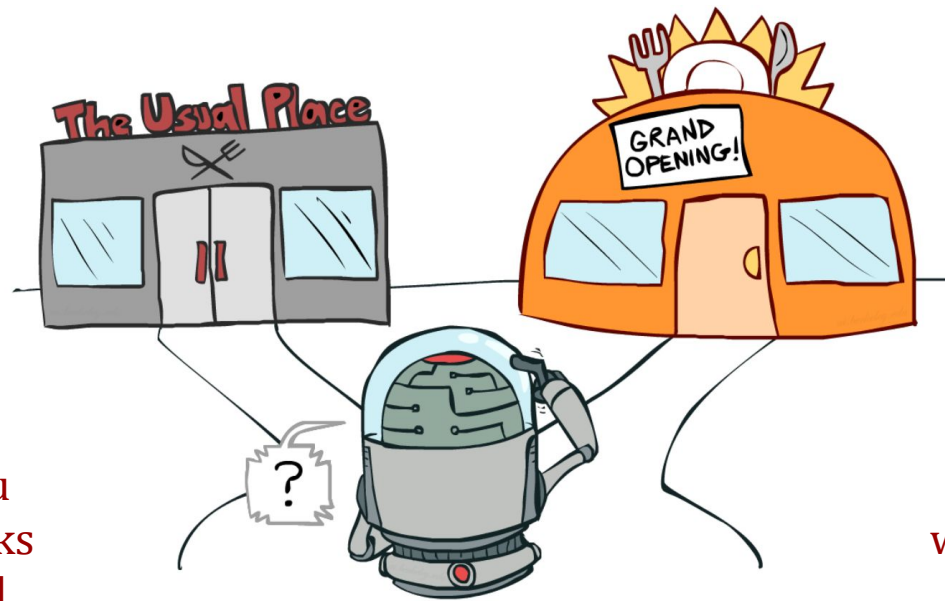


Exploration/exploitation

Exploitation:
do something you
already know works
(reasonably) well



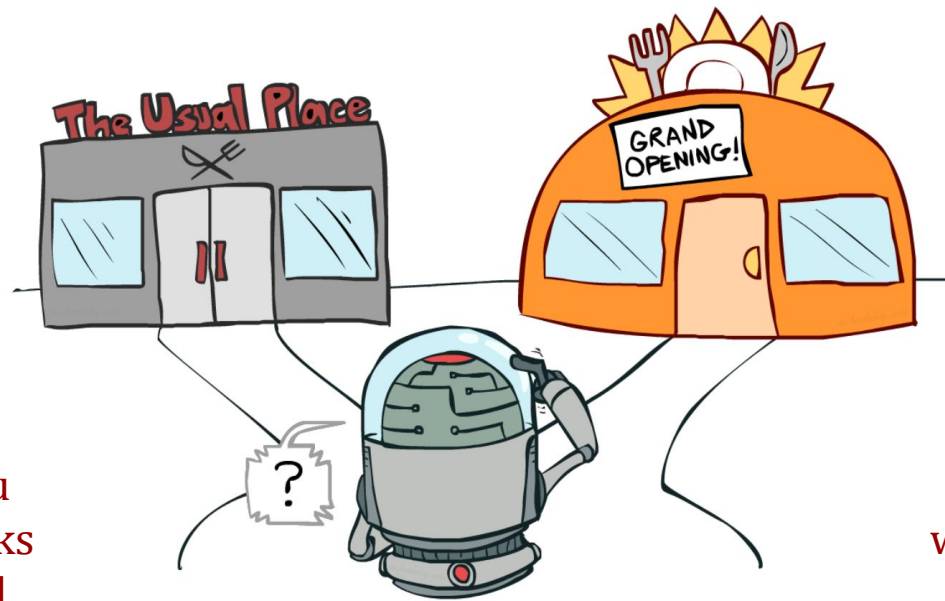
Exploration/exploitation



Exploitation:
do something you
already know works
(reasonably) well

Exploration:
try something new
which may work even
better (or worse)

Exploration/exploitation



Exploitation:
do something you
already know works
(reasonably) well

Exploration:
try something new
which may work even
better (or worse)

Fundamental trade-off in all decision-making problems (bandits, RL)
(and in life in general)

Bandit algorithm pseudocode

Initialization: Initialize policy $\pi(a)$

for $t = 1 \dots T$ **do**

$a_t \sim \pi(a)$

$r_t \sim p(r|a_t)$

 Update π based on (a_t, r_t)

end

/* Sample from policy */

/* Observe reward */

Bandit algorithm pseudocode

Initialization: Initialize policy $\pi(a)$

for $t = 1 \dots T$ **do**

$a_t \sim \pi(a)$

$r_t \sim p(r|a_t)$

 Update π based on (a_t, r_t)

end

/* Sample from policy */

/* Observe reward */

Bandit algorithm pseudocode

Initialization: Initialize policy $\pi(a)$

for $t = 1 \dots T$ **do**

$a_t \sim \pi(a)$

$r_t \sim p(r|a_t)$

 Update π based on (a_t, r_t)

end

/* Sample from policy */

/* Observe reward */

Exploration/exploitation!

Bandit algorithm pseudocode

Initialization: Initialize policy $\pi(a)$

for $t = 1 \dots T$ **do**

$a_t \sim \pi(a)$
 $r_t \sim p(r|a_t)$
 Update π based on (a_t, r_t)

end

/* Sample from policy */
/* Observe reward */

Bandit algorithm pseudocode

Initialization: Initialize policy $\pi(a)$

for $t = 1 \dots T$ **do**

$a_t \sim \pi(a)$

$r_t \sim p(r|a_t)$

 Update π based on (a_t, r_t)

end

/* Sample from policy */

/* Observe reward */

Bandit algorithm pseudocode

Initialization: Initialize policy $\pi(a)$

for $t = 1 \dots T$ **do**

$a_t \sim \pi(a)$

$r_t \sim p(r|a_t)$

 Update π based on (a_t, r_t)

end

/* Sample from policy */

/* Observe reward */

Exploration/exploitation!

Bandit algorithm pseudocode

Initialization: Initialize policy $\pi(a)$

for $t = 1 \dots T$ **do**

$a_t \sim \pi(a)$
 $r_t \sim p(r|a_t)$
 Update π based on (a_t, r_t)

end

/* Sample from policy */
/* Observe reward */

Bandit algorithm pseudocode

Initialization: Initialize policy $\pi(a)$

for $t = 1 \dots T$ **do**

$a_t \sim \pi(a)$

$r_t \sim p(r|a_t)$

 Update π based on (a_t, r_t)

end

/* Sample from policy */
/* Observe reward */

etc.

Part 3:

Updating a mean

Estimate a mean

Many algorithms rely on our ability to estimate the *mean* reward of an action:

Estimate a mean

Many algorithms rely on our ability to estimate the *mean* reward of an action:

$$Q_n = \frac{r_1 + r_2 + \dots + r_n}{n} = \frac{1}{n} \sum_{i=1}^n r_i$$

Estimate a mean

Many algorithms rely on our ability to estimate the *mean* reward of an action:

$$Q_n = \frac{r_1 + r_2 + \dots + r_n}{n} = \frac{1}{n} \sum_{i=1}^n r_i$$

Q: For arm a you observe $r_1=4$, $r_2=7$, $r_3=1$. What is $Q(a)$?

Estimate a mean

Many algorithms rely on our ability to estimate the *mean* reward of an action:

$$Q_n = \frac{r_1 + r_2 + \dots + r_n}{n} = \frac{1}{n} \sum_{i=1}^n r_i$$

Q: For arm a you observe $r_1=4$, $r_2=7$, $r_3=1$. What is $Q(a)$?

A: $(4 + 7 + 1) / 3 = 4$

Estimate a mean

Many algorithms rely on our ability to estimate the *mean* reward of an action:

$$Q_n = \frac{r_1 + r_2 + \dots + r_n}{n} = \frac{1}{n} \sum_{i=1}^n r_i$$

However, now next r_{n+1} comes in, and *how do we update the current mean Q_n ?*

- 1) Incremental update
- 2) Learning update

Incremental mean update

*Can we write Q_n as a function of the **previous mean** Q_{n-1} and the **new reward** r_n ?*

Incremental mean update

$$Q_n = \frac{1}{n} \sum_{i=1}^n r_i$$

Incremental mean update

$$\begin{aligned} Q_n &= \frac{1}{n} \sum_{i=1}^n r_i \\ &= \frac{1}{n} \left[r_n + \sum_{i=1}^{n-1} r_i \right] \end{aligned}$$

Incremental mean update

$$\begin{aligned}Q_n &= \frac{1}{n} \sum_{i=1}^n r_i \\&= \frac{1}{n} \left[r_n + \sum_{i=1}^{n-1} r_i \right] \\&= \frac{1}{n} \left[r_n + (n-1) \frac{1}{(n-1)} \sum_{i=1}^{n-1} r_i \right]\end{aligned}$$

Incremental mean update

$$\begin{aligned}Q_n &= \frac{1}{n} \sum_{i=1}^n r_i \\&= \frac{1}{n} \left[r_n + \sum_{i=1}^{n-1} r_i \right] \\&= \frac{1}{n} \left[r_n + (n-1) \frac{1}{(n-1)} \sum_{i=1}^{n-1} r_i \right] \\&= \frac{1}{n} \left[r_n + (n-1) Q_{n-1} \right]\end{aligned}$$

Incremental mean update

$$\begin{aligned}Q_n &= \frac{1}{n} \sum_{i=1}^n r_i \\&= \frac{1}{n} \left[r_n + \sum_{i=1}^{n-1} r_i \right] \\&= \frac{1}{n} \left[r_n + (n-1) \frac{1}{(n-1)} \sum_{i=1}^{n-1} r_i \right] \\&= \frac{1}{n} \left[r_n + (n-1) Q_{n-1} \right] \\&= \frac{1}{n} \left[r_n + n \cdot Q_{n-1} - Q_{n-1} \right]\end{aligned}$$

Incremental mean update

$$\begin{aligned}Q_n &= \frac{1}{n} \sum_{i=1}^n r_i \\&= \frac{1}{n} \left[r_n + \sum_{i=1}^{n-1} r_i \right] \\&= \frac{1}{n} \left[r_n + (n-1) \frac{1}{(n-1)} \sum_{i=1}^{n-1} r_i \right] \\&= \frac{1}{n} \left[r_n + (n-1) Q_{n-1} \right] \\&= \frac{1}{n} \left[r_n + n \cdot Q_{n-1} - Q_{n-1} \right] \\Q_n &= Q_{n-1} + \frac{1}{n} [r_n - Q_{n-1}]\end{aligned}$$

Incremental mean update

$$\begin{aligned}Q_n &= \frac{1}{n} \sum_{i=1}^n r_i \\&= \frac{1}{n} \left[r_n + \sum_{i=1}^{n-1} r_i \right] \\&= \frac{1}{n} \left[r_n + (n-1) \frac{1}{(n-1)} \sum_{i=1}^{n-1} r_i \right] \\&= \frac{1}{n} \left[r_n + (n-1) Q_{n-1} \right] \\&= \frac{1}{n} \left[r_n + n \cdot Q_{n-1} - Q_{n-1} \right] \\Q_n &= Q_{n-1} + \frac{1}{n} [r_n - Q_{n-1}]\end{aligned}$$

**Not for lecture:
check this
derivation in the
book/lecture slides!**

Incremental mean update

$$Q_n = Q_{n-1} + \frac{1}{n}[r_n - Q_{n-1}]$$

**Incremental
update rule for
the mean**

Incremental mean update

Question:

- For action 1 we took 3 samples so far, with a mean reward of 4.0
- We take a new sample, and observe a reward of 6.0
- Compute the new $Q(a)$.

$$Q_n = Q_{n-1} + \frac{1}{n} [r_n - Q_{n-1}]$$

**Incremental
update rule for
the mean**

Incremental mean update

Question:

- For action 1 we took 3 samples so far, with a mean reward of 4.0
- We take a new sample, and observe a reward of 6.0
- Compute the new $Q(a)$.

Answer: $4.0 + \frac{1}{4} * [6.0 - 4.0] = 4.5$

$$Q_n = Q_{n-1} + \frac{1}{n} [r_n - Q_{n-1}]$$

**Incremental
update rule for
the mean**

Learning mean update

'Simply move the new mean a bit in the direction of the last observed reward'

Learning mean update

'Simply move the new mean a bit in the direction of the last observed reward'

$$Q_n \leftarrow Q_{n-1} + \alpha [r_n - Q_{n-1}]$$

Learning mean update

'Simply move the new mean a bit in the direction of the last observed reward'

$$Q_n \leftarrow Q_{n-1} + \alpha [r_n - Q_{n-1}]$$



Learning rate α

Learning mean update

'Simply move the new mean a bit in the direction of the last observed reward'

$$Q_n \leftarrow Q_{n-1} + \alpha [r_n - Q_{n-1}]$$

Question:

- For action 1 we took 3 samples so far, with a mean reward of 4.0
- We take a new sample, and observe a reward of 6.0
- Compute the new $Q(a)$ for learning rate = 0.2

Learning mean update

'Simply move the new mean a bit in the direction of the last observed reward'

$$Q_n \leftarrow Q_{n-1} + \alpha [r_n - Q_{n-1}]$$

Question:

- For action 1 we took 3 samples so far, with a mean reward of 4.0
- We take a new sample, and observe a reward of 6.0
- Compute the new $Q(a)$ for learning rate = 0.2

Answer: $4.0 + 0.2 * [6.0 - 4.0] = 4.4$

Summary: update of mean

- *Incremental update:*

- *Learning update:*

Summary: update of mean

- *Incremental update:*

$$Q_n = Q_{n-1} + \frac{1}{n} [r_n - Q_{n-1}]$$

- *Learning update:*

$$Q_n = Q_{n-1} + \alpha [r_n - Q_{n-1}]$$

Summary: update of mean

- *Incremental update:*

$$Q_n = Q_{n-1} + \boxed{\frac{1}{n}} [r_n - Q_{n-1}]$$

- *Learning update:*

$$Q_n = Q_{n-1} + \boxed{\alpha} [r_n - Q_{n-1}]$$

Summary: update of mean

- *Incremental update:*

$$Q_n = Q_{n-1} + \boxed{\frac{1}{n}} [r_n - Q_{n-1}]$$

Equally weights each sample

- *Learning update:*

$$Q_n = Q_{n-1} + \boxed{\alpha} [r_n - Q_{n-1}]$$

More weight on recent samples

Summary: update of mean

- *Incremental update:*

$$Q_n = Q_{n-1} + \boxed{\frac{1}{n}} [r_n - Q_{n-1}]$$

- + **Unbiased initial estimates**
- **Not for non-stationary problems**

- *Learning update:*

$$Q_n = Q_{n-1} + \boxed{\alpha} [r_n - Q_{n-1}]$$

- + **Also works in non-stationary problems**
- **Biased initial estimates (only make small step)**

Summary: update of mean

(Book discusses a way to anneal from incremental to learning update - combine both benefits)

- *Incremental update:*

$$Q_n = Q_{n-1} + \boxed{\frac{1}{n}} [r_n - Q_{n-1}]$$

- + **Unbiased initial estimates**
- **Not for non-stationary problems**

- *Learning update:*

$$Q_n = Q_{n-1} + \boxed{\alpha} [r_n - Q_{n-1}]$$

- + **Also works in non-stationary problems**
- **Biased initial estimates (only make small step)**

Break

Part 4:

Bandit algorithms

Bandit pseudocode

Bandit pseudocode:

```
Initialization: Initialize policy  $\pi(a)$   
for  $t = 1 \dots T$  do  
     $a_t \sim \pi(a)$   
     $r_t \sim p(r|a_t)$   
    Update  $\pi$  based on  $(a_t, r_t)$   
end
```

```
/* Sample from policy */  
/* Observe reward */
```

Bandit pseudocode

Bandit pseudocode:

```
Initialization: Initialize policy  $\pi(a)$   
for  $t = 1 \dots T$  do  
     $a_t \sim \pi(a)$   
     $r_t \sim p(r|a_t)$   
    Update  $\pi$  based on  $(a_t, r_t)$   
end
```

```
/* Sample from policy */  
/* Observe reward */
```

Bandit pseudocode

Bandit pseudocode:

```
Initialization: Initialize policy  $\pi(a)$ 
for  $t = 1 \dots T$  do
     $a_t \sim \pi(a)$ 
     $r_t \sim p(r|a_t)$ 
    Update  $\pi$  based on  $(a_t, r_t)$ 
end
```

/* Sample from policy */
/* Observe reward */

Three main things we need to decide on:

1. Initialization
2. Action selection (exploration/exploitation)
3. Updating

Bandit pseudocode

Bandit pseudocode:

```
Initialization: Initialize policy  $\pi(a)$ 
for  $t = 1 \dots T$  do
     $a_t \sim \pi(a)$ 
     $r_t \sim p(r|a_t)$ 
    Update  $\pi$  based on  $(a_t, r_t)$ 
end
```

/* Sample from policy */
/* Observe reward */

Three main things we need to decide on:

1. Initialization
2. Action selection (exploration/exploitation)
3. **Updating**

Before the break we already discussed two ways to update a mean estimate

Bandit pseudocode

Bandit pseudocode:

```
Initialization: Initialize policy  $\pi(a)$ 
for  $t = 1 \dots T$  do
     $a_t \sim \pi(a)$  /* Sample from policy */
     $r_t \sim p(r|a_t)$  /* Observe reward */
    Update  $\pi$  based on  $(a_t, r_t)$ 
end
```

Three main things we need to decide on:

1. Initialization
2. **Action selection (exploration/exploitation)**
3. Updating

Action selection: exploitation

Exploitation:

Action selection: exploitation

Exploitation:

select action with current highest mean estimate

$$\pi_{\text{greedy}}(a) = f(Q) = \begin{cases} 1, & \text{if } a = \arg \max_{b \in \mathcal{A}} Q(b) \\ 0, & \text{otherwise} \end{cases}$$

Action selection: exploitation

Exploitation:

select action with current highest mean estimate

$$\pi_{\text{greedy}}(a) = f(Q) = \begin{cases} 1, & \text{if } a = \arg \max_{b \in \mathcal{A}} Q(b) \\ 0, & \text{otherwise} \end{cases}$$

Example:

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

Action selection: exploitation

Exploitation:

select action with current highest mean estimate

$$\pi_{\text{greedy}}(a) = f(Q) = \begin{cases} 1, & \text{if } a = \arg \max_{b \in \mathcal{A}} Q(b) \\ 0, & \text{otherwise} \end{cases}$$

Example:

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

Question: what probability will each action get?

Action selection: exploitation

Exploitation:

select action with current highest mean estimate

$$\pi_{\text{greedy}}(a) = f(Q) = \begin{cases} 1, & \text{if } a = \arg \max_{b \in \mathcal{A}} Q(b) \\ 0, & \text{otherwise} \end{cases}$$

Example:

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

Answer:

$$\pi(a=1) = 0.0$$

$$\pi(a=2) = 0.0$$

$$\pi(a=3) = 0.0$$

$$\pi(a=4) = 1.0$$

Question: what probability will each action get?

Action selection: exploitation

Exploitation:

select action with current highest mean estimate

$$\pi_{\text{greedy}}(a) = f(Q) = \begin{cases} 1, & \text{if } a = \arg \max_{b \in \mathcal{A}} Q(b) \\ 0, & \text{otherwise} \end{cases}$$

Example:

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

“Greedy policy”

Question: what probability will each action get?

Action selection: exploration

We need to introduce exploration

Action selection: exploration

We need to introduce exploration

Discuss three possible approaches:

1. Random perturbation (ϵ -greedy)
2. Optimistic initialization (oi)
3. Uncertainty-based (ucb)

Action selection: exploration

We need to introduce exploration

Discuss three possible approaches:

1. **Random perturbation** (ϵ -greedy)
2. Optimistic initialization (oi)
3. Uncertainty-based (ucb)

ϵ -greedy

“Act greedily, but with (small) probability ϵ , sample a random other action”

ϵ -greedy

“Act greedily, but with (small) probability ϵ , sample a random other action”

$$\pi_{\epsilon\text{-greedy}}(a) = f(Q, \epsilon) = \begin{cases} 1 - \epsilon, & \text{if } a = \arg \max_{b \in \mathcal{A}} Q(b) \\ \frac{\epsilon}{(|\mathcal{A}|-1)}, & \text{otherwise} \end{cases}$$

ϵ -greedy

“Act greedily, but with (small) probability ϵ , sample a random other action”

$$\pi_{\epsilon\text{-greedy}}(a) = f(Q, \epsilon) = \begin{cases} 1 - \epsilon, & \text{if } a = \arg \max_{b \in \mathcal{A}} Q(b) \\ \frac{\epsilon}{(|\mathcal{A}|-1)}, & \text{otherwise} \end{cases}$$

ϵ = exploration parameter

scale the amount of exploration

ϵ -greedy

“Act greedily, but with (small) probability ϵ , sample a random other action”

$$\pi_{\epsilon\text{-greedy}}(a) = f(Q, \epsilon) = \begin{cases} 1 - \epsilon, & \text{if } a = \arg \max_{b \in \mathcal{A}} Q(b) \\ \frac{\epsilon}{(|\mathcal{A}|-1)}, & \text{otherwise} \end{cases}$$

Example

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

ϵ -greedy

“Act greedily, but with (small) probability ϵ , sample a random other action”

$$\pi_{\epsilon\text{-greedy}}(a) = f(Q, \epsilon) = \begin{cases} 1 - \epsilon, & \text{if } a = \arg \max_{b \in \mathcal{A}} Q(b) \\ \frac{\epsilon}{(|\mathcal{A}|-1)}, & \text{otherwise} \end{cases}$$

Example

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

Question: Assume $\epsilon = 0.15$. What probability of selection will each action get?

ϵ -greedy

“Act greedily, but with (small) probability ϵ , sample a random other action”

$$\pi_{\epsilon\text{-greedy}}(a) = f(Q, \epsilon) = \begin{cases} 1 - \epsilon, & \text{if } a = \arg \max_{b \in \mathcal{A}} Q(b) \\ \frac{\epsilon}{(|\mathcal{A}|-1)}, & \text{otherwise} \end{cases}$$

Example

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

Answer:

$$\pi(a=1) = 0.05$$

$$\pi(a=2) = 0.05$$

$$\pi(a=3) = 0.05$$

$$\pi(a=4) = 0.85$$

Question: Assume $\epsilon = 0.15$. What probability of selection will each action get?

ϵ -greedy

Also fine if you take ϵ the probability of selecting any action randomly (not just the non-greedy ones)

“Act greedily, but with (small) probability ϵ , sample a random other action”

$$a_t = \begin{cases} \arg \max_{a \in \mathcal{A}} Q(a) & \text{with } p = 1 - \epsilon \\ \text{random,} & \text{otherwise} \end{cases}$$

Example

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

ϵ -greedy

Also fine if you take ϵ the probability of selecting any action randomly (not just the non-greedy ones)

“Act greedily, but with (small) probability ϵ , sample a random other action”

$$a_t = \begin{cases} \arg \max_{a \in \mathcal{A}} Q(a) & \text{with } p = 1 - \epsilon \\ \text{random,} & \text{otherwise} \end{cases}$$

Example

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

Question: Use this new definition with $\epsilon=0.20$. What probability of selection will each action now get?

ϵ -greedy

Also fine if you take ϵ the probability of selecting any action randomly (not just the non-greedy ones)

“Act greedily, but with (small) probability ϵ , sample a random other action”

$$a_t = \begin{cases} \arg \max_{a \in \mathcal{A}} Q(a) & \text{with } p = 1 - \epsilon \\ \text{random,} & \text{otherwise} \end{cases}$$

Example

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

Answer:

$$\pi(a=1) = 0.05$$

$$\pi(a=2) = 0.05$$

$$\pi(a=3) = 0.05$$

$$\pi(a=4) = 0.85$$

Question: Use this new definition with $\epsilon=0.20$. What probability of selection will each action now get?

ϵ -greedy

Also fine if you take ϵ the probability of selecting any action randomly (not just the non-greedy ones)

“Act greedily, but with (small) probability ϵ , sample a random other action”

$$a_t = \begin{cases} \arg \max_{a \in \mathcal{A}} Q(a) & \text{with } p = 1 - \epsilon \\ \text{random,} & \text{otherwise} \end{cases}$$

Example

These two implementations are equivalent (the interpretation of ϵ just changes)

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

Question: Use this new definition with $\epsilon=0.20$. What probability of selection will each action now get?

Answer:

$$\pi(a=1) = 0.05$$

$$\pi(a=2) = 0.05$$

$$\pi(a=3) = 0.05$$

$$\pi(a=4) = 0.85$$

ϵ -greedy

“Act greedily, but with (small) probability ϵ , sample a random other action”

$$a_t = \begin{cases} \arg \max_{a \in \mathcal{A}} Q(a) & \text{with } p = 1 - \epsilon \\ \text{random,} & \text{otherwise} \end{cases}$$

Example

$Q(a = 1)$	$Q(a = 2)$	$Q(a = 3)$	$Q(a = 4)$
1.2	0.3	-2.4	3.5

Final note: if multiple actions have the same highest Q-value you *randomly* break ties between them

ϵ -greedy pseudocode

Algorithm 2: ϵ -greedy bandit algorithm.

Input: Exploration parameter $\epsilon \in [0, 1]$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = 0$, $n(a) = 0 \forall a \in \mathcal{A}$

for $t = 1 \dots T$ **do**

$a_t = \begin{cases} \arg \max_{a \in \mathcal{A}} Q(a) & \text{with } p = 1 - \epsilon \\ \text{random,} & \text{otherwise} \end{cases}$ */* ϵ -greedy action */*

$r_t \sim p(r|a_t)$ */* Sample reward */*

$n(a_t) \leftarrow n(a_t) + 1$ */* Update count */*

$Q(a_t) \leftarrow Q(a_t) + \frac{1}{n(a_t)} [r_t - Q(a_t)]$ */* Incr. update mean */*

end

Algorithm now requires an input ϵ

ϵ -greedy pseudocode

Algorithm 2: ϵ -greedy bandit algorithm.

Input: Exploration parameter $\epsilon \in [0, 1]$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = 0, n(a) = 0 \forall a \in \mathcal{A}$

for $t = 1 \dots T$ **do**

$a_t = \begin{cases} \arg \max_{a \in \mathcal{A}} Q(a) & \text{with } p = 1 - \epsilon \\ \text{random,} & \text{otherwise} \end{cases}$ */* ϵ -greedy action */*

$r_t \sim p(r|a_t)$ */* Sample reward */*

$n(a_t) \leftarrow n(a_t) + 1$ */* Update count */*

$Q(a_t) \leftarrow Q(a_t) + \frac{1}{n(a_t)} [r_t - Q(a_t)]$ */* Incr. update mean */*

end

Initialize means and counts to 0

ϵ -greedy pseudocode

Algorithm 2: ϵ -greedy bandit algorithm.

Input: Exploration parameter $\epsilon \in [0, 1]$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = 0$, $n(a) = 0 \forall a \in \mathcal{A}$

for $t = 1 \dots T$ **do**

$$a_t = \begin{cases} \arg \max_{a \in \mathcal{A}} Q(a) & \text{with } p = 1 - \epsilon \\ \text{random,} & \text{otherwise} \end{cases} \quad \text{/* } \epsilon\text{-greedy action */}$$

$r_t \sim p(r|a_t)$ /* Sample reward */

$n(a_t) \leftarrow n(a_t) + 1$ /* Update count */

$Q(a_t) \leftarrow Q(a_t) + \frac{1}{n(a_t)} [r_t - Q(a_t)]$ /* Incr. update mean */

end

Select ϵ -greedy action

ϵ -greedy pseudocode

Algorithm 2: ϵ -greedy bandit algorithm.

Input: Exploration parameter $\epsilon \in [0, 1]$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = 0$, $n(a) = 0 \forall a \in \mathcal{A}$

for $t = 1 \dots T$ **do**

$a_t = \begin{cases} \arg \max_{a \in \mathcal{A}} Q(a) & \text{with } p = 1 - \epsilon \\ \text{random,} & \text{otherwise} \end{cases}$ */* ϵ -greedy action */*

$r_t \sim p(r|a_t)$ */* Sample reward */*

$n(a_t) \leftarrow n(a_t) + 1$ */* Update count */*

$Q(a_t) \leftarrow Q(a_t) + \frac{1}{n(a_t)} [r_t - Q(a_t)]$ */* Incr. update mean */*

end

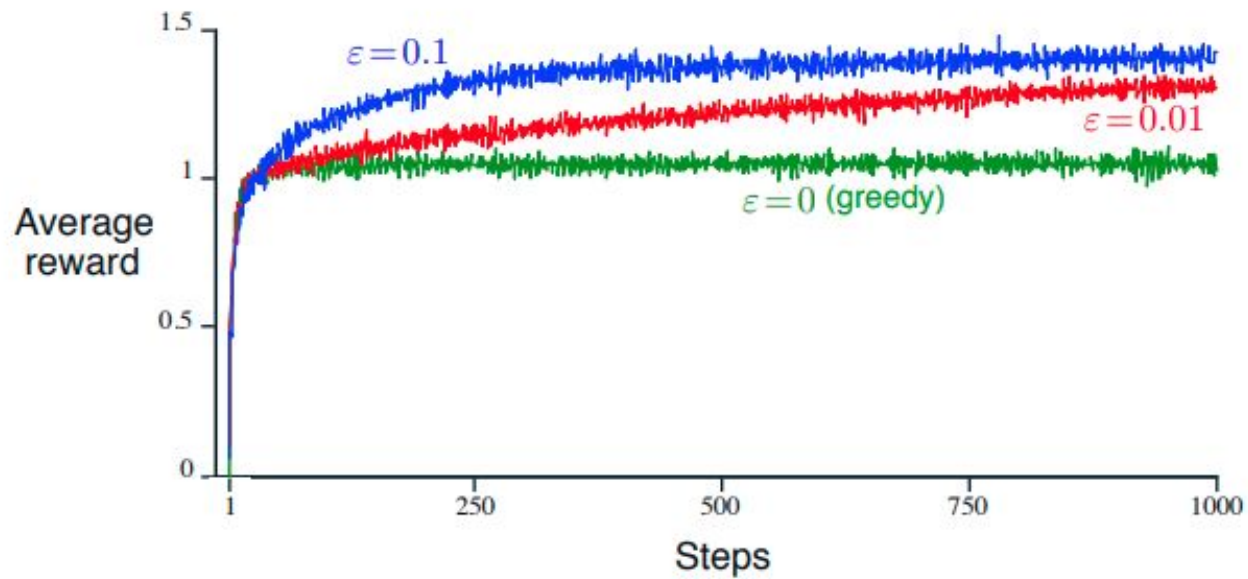
Incremental update to track the means

ϵ -greedy performance

Exploration parameters should usually be neither too high nor too low

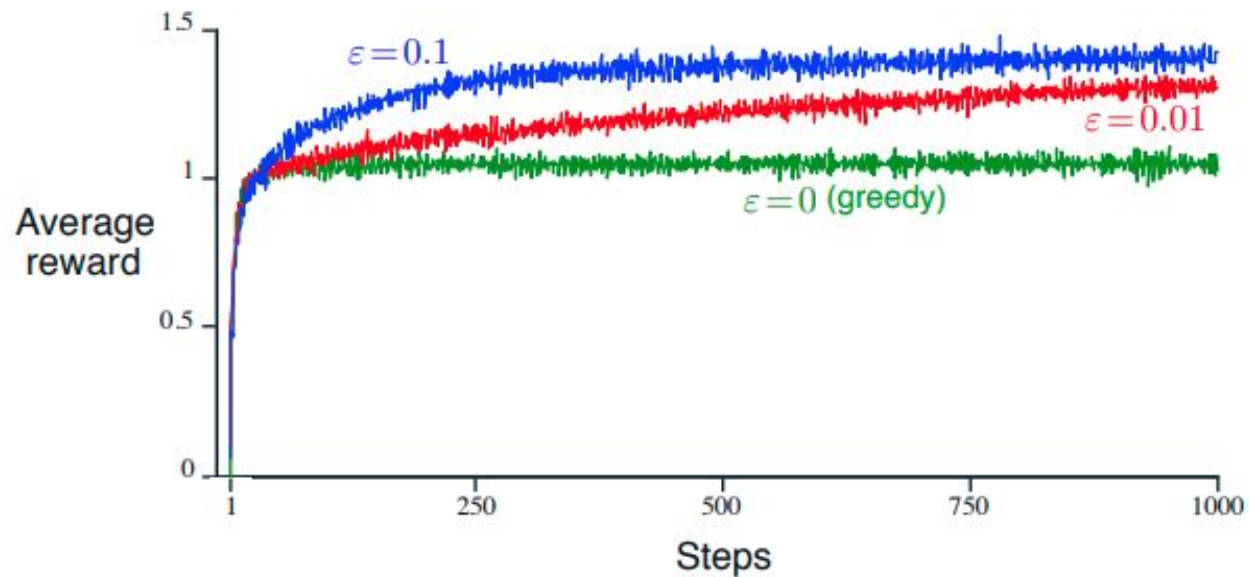
ϵ -greedy performance

Exploration parameters should usually be neither too high nor too low



ϵ -greedy performance

Exploration parameters should usually be neither too high nor too low



You will implement such an experiment in the assignments!

ε decay

ϵ decay

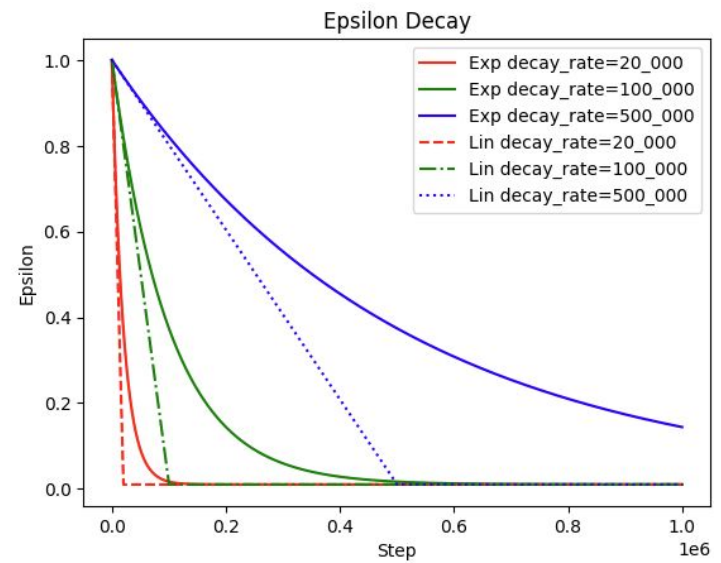
Vanilla ϵ -greedy always has fixed exploration and exploitation probabilities, while we want to transition from exploration to exploitation!

ϵ decay

Vanilla e-greedy always has fixed exploration and exploitation probabilities, while we want to transition from exploration to exploitation!

Potential solution:

- decay epsilon during training from high (1.0) to low (~ 0.0)



ϵ decay

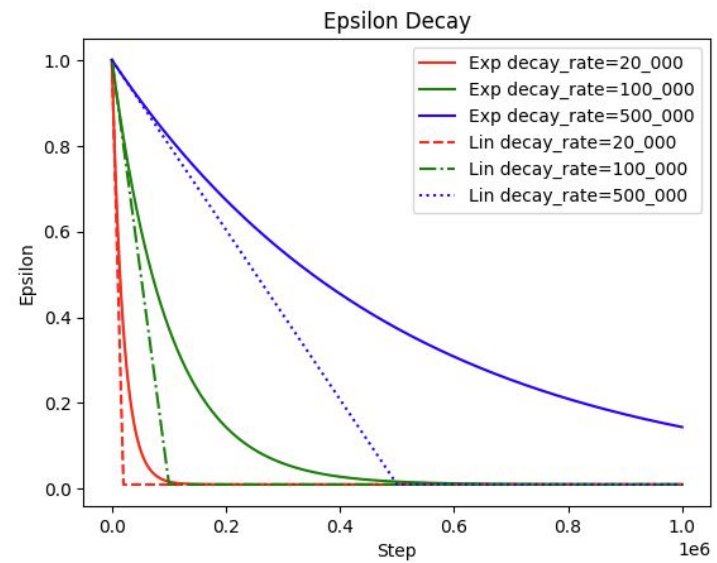
Vanilla ϵ -greedy always has fixed exploration and exploitation probabilities, while we want to transition from exploration to exploitation!

Potential solution:

- decay epsilon during training from high (1.0) to low (~ 0.0)

Remaining downside:

- Trade-off is not adaptively based on data (i.e., we need to tune extra hyperparameters)



Action selection: exploration

We need to introduce exploration

Discuss three possible approaches:

1. Random perturbation (ϵ -greedy)
2. **Optimistic initialization** (oi)
3. Uncertainty-based (ucb)

Optimistic initialization

For learning based updates of the mean, the initial value really matters:

But: a higher initial value should also encourage exploration!

Optimistic initialization

For learning based updates of the mean, the initial value really matters:

But: a higher initial value should also encourage exploration!

Idea:

- Initialize optimistic mean estimates
- Select actions greedily
- Learning-based update of the mean

Optimistic initialization

Algorithm 3: Optimistic initialization with greedy action selection bandit algorithm.

Input: Initial value $\psi \in \mathbb{R}$, learning rate $\eta \in \mathbb{R}^+$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = \psi \ \forall a \in \mathcal{A}$ /* Optimistic init. */

for $t = 1..T$ **do**
$$a_t = \arg \max_{a \in \mathcal{A}} Q(a)$$

```
/* Sample greedy action */
```

$$r_t \sim p(r|a_t)$$

```
/* Sample reward */
```

$$Q(a_t) \leftarrow Q(a_t) + \eta \cdot [r_t - Q(a_t)]$$

```
/* Learning update mean */
```

end

Optimistic initialization

Algorithm 3: Optimistic initialization with greedy action selection bandit algorithm.

Input: Initial value $\psi \in \mathbb{R}$, learning rate $\eta \in \mathbb{R}^+$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = \psi \forall a \in \mathcal{A}$ /* Optimistic init. */
for $t = 1 \dots T$ **do**
 $a_t = \arg \max_{a \in \mathcal{A}} Q(a)$ /* Sample greedy action */
 $r_t \sim p(r|a_t)$ /* Sample reward */
 $Q(a_t) \leftarrow Q(a_t) + \eta \cdot [r_t - Q(a_t)]$ /* Learning update mean */
end

Algorithm now requires an initial value ψ for each action
(= exploration parameter) + a learning rate η

Optimistic initialization

Algorithm 3: Optimistic initialization with greedy action selection bandit algorithm.

Input: Initial value $\psi \in \mathbb{R}$, learning rate $\eta \in \mathbb{R}^+$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = \psi \forall a \in \mathcal{A}$ /* Optimistic init. */

for $t = 1 \dots T$ **do**

$a_t = \arg \max_{a \in \mathcal{A}} Q(a)$ /* Sample greedy action */

$r_t \sim p(r|a_t)$ /* Sample reward */

$Q(a_t) \leftarrow Q(a_t) + \eta \cdot [r_t - Q(a_t)]$ /* Learning update mean */

end

Initialize all mean estimates to ψ

Optimistic initialization

Algorithm 3: Optimistic initialization with greedy action selection bandit algorithm.

Input: Initial value $\psi \in \mathbb{R}$, learning rate $\eta \in \mathbb{R}^+$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = \psi \forall a \in \mathcal{A}$ /* Optimistic init. */

for $t = 1 \dots T$ **do**

$a_t = \arg \max_{a \in \mathcal{A}} Q(a)$

 /* Sample greedy action */

$r_t \sim p(r|a_t)$

 /* Sample reward */

$Q(a_t) \leftarrow Q(a_t) + \eta \cdot [r_t - Q(a_t)]$

 /* Learning update mean */

end

Greedily select next action

Optimistic initialization

Algorithm 3: Optimistic initialization with greedy action selection bandit algorithm.

Input: Initial value $\psi \in \mathbb{R}$, learning rate $\eta \in \mathbb{R}^+$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = \psi \forall a \in \mathcal{A}$ */* Optimistic init. */*

for $t = 1 \dots T$ **do**

$a_t = \arg \max_{a \in \mathcal{A}} Q(a)$

/ Sample greedy action */*

$r_t \sim p(r|a_t)$

/ Sample reward */*

$Q(a_t) \leftarrow Q(a_t) + \eta \cdot [r_t - Q(a_t)]$

/ Learning update mean */*

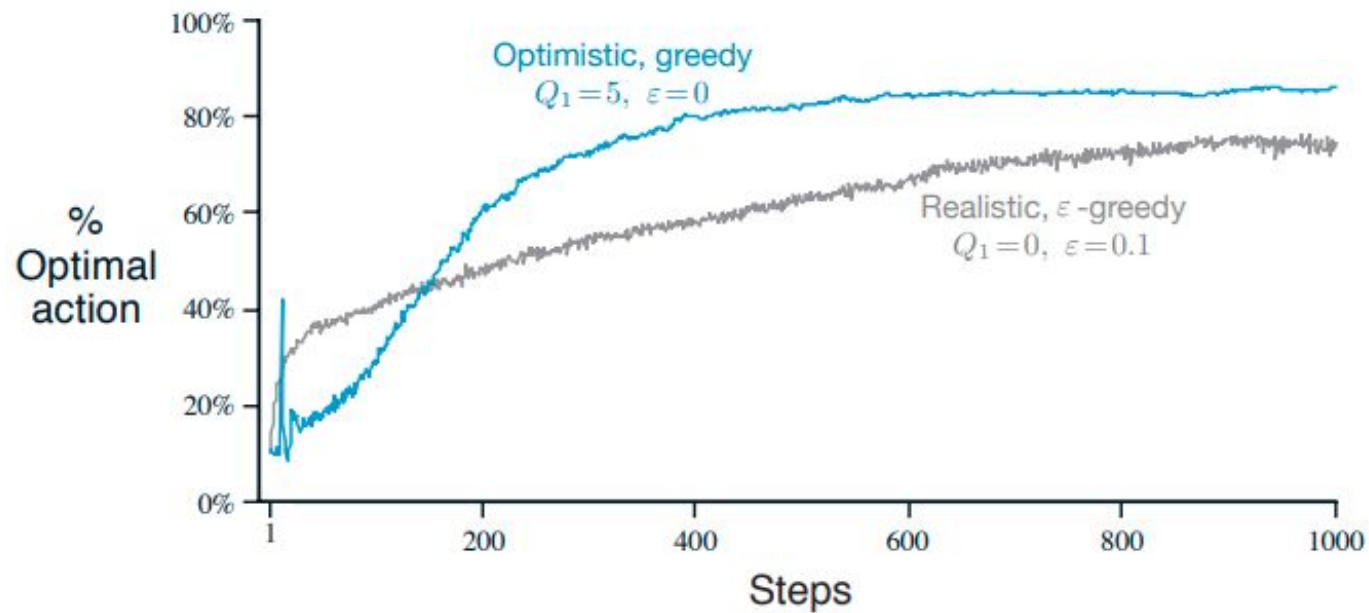
end

Learning update of means

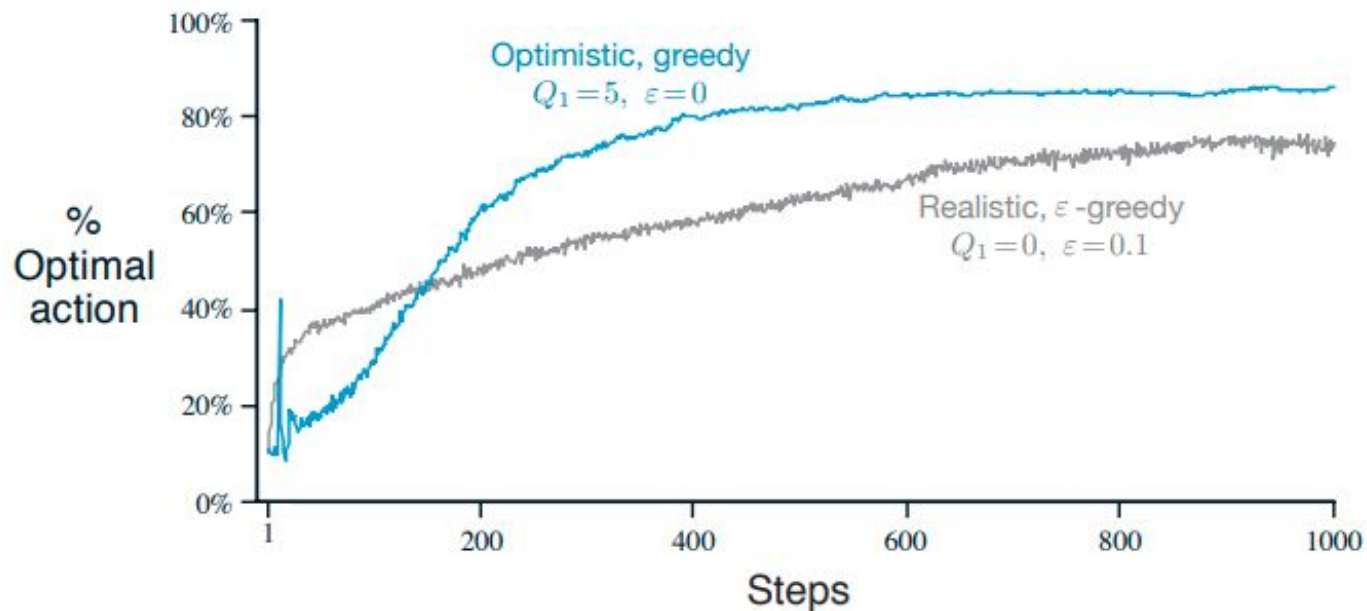
Optimistic initialization performance



Optimistic initialization performance



Optimistic initialization performance



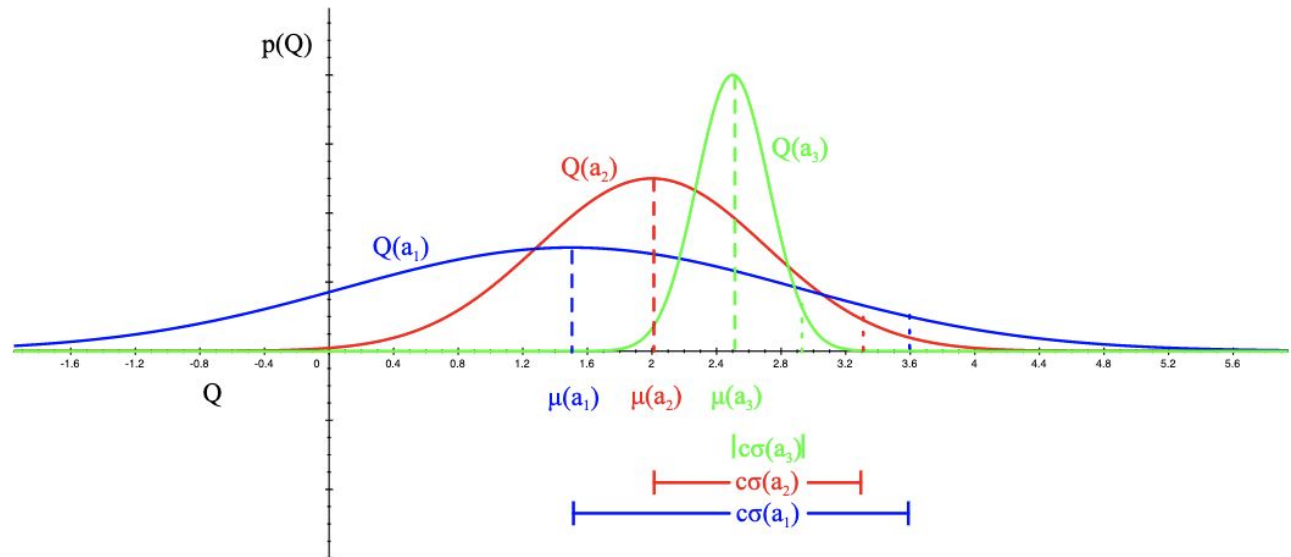
You will also experiment with different initial values in the assignment

Optimistic initialization performance

Question:

For the below bandit, what would be a good optimistic initial value?

- A) 0
- B) 2
- C) 8
- D) 1000

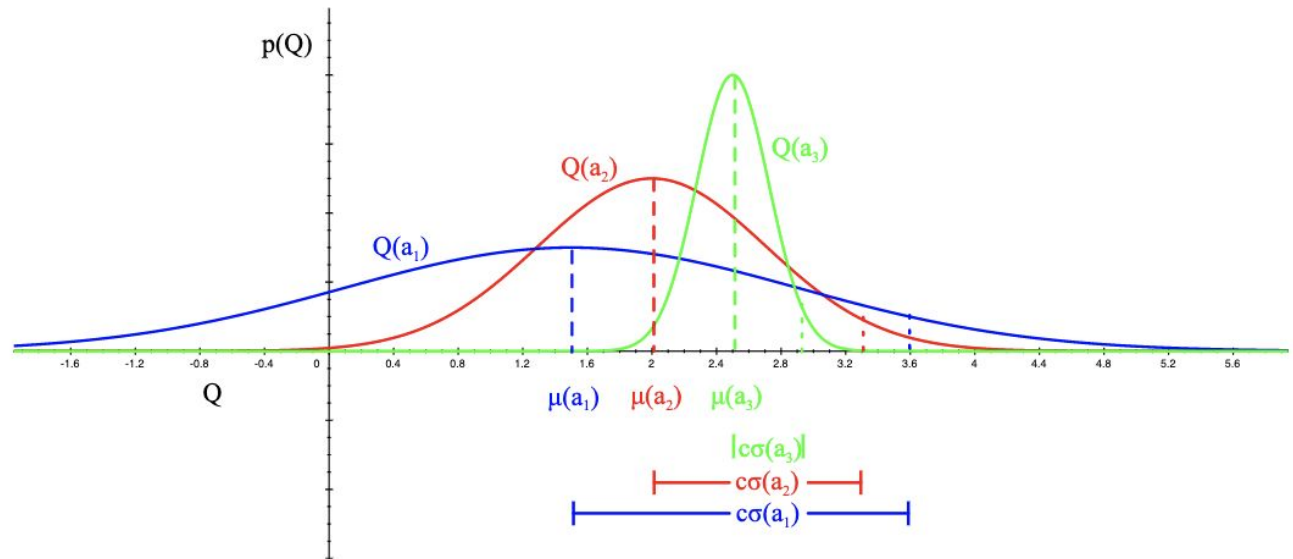


Optimistic initialization performance

Question:

For the below bandit, what would be a good optimistic initial value?

- A) 0
- B) 2
- C) 8**
- D) 1000

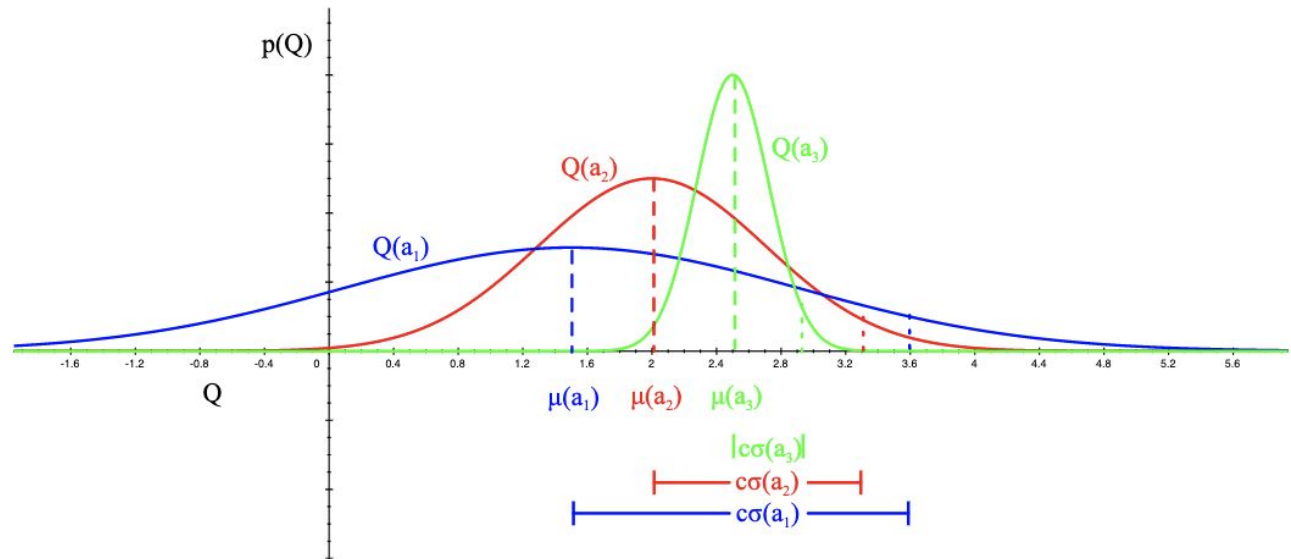


Optimistic initialization performance

Question:

For the below bandit, what would be a good optimistic initial value?

- A) 0
- B) 2
- C) 8**
- D) 1000



Put optimistic init
neither too high
nor too low

Action selection: exploration

We need to introduce exploration

Discuss three possible approaches:

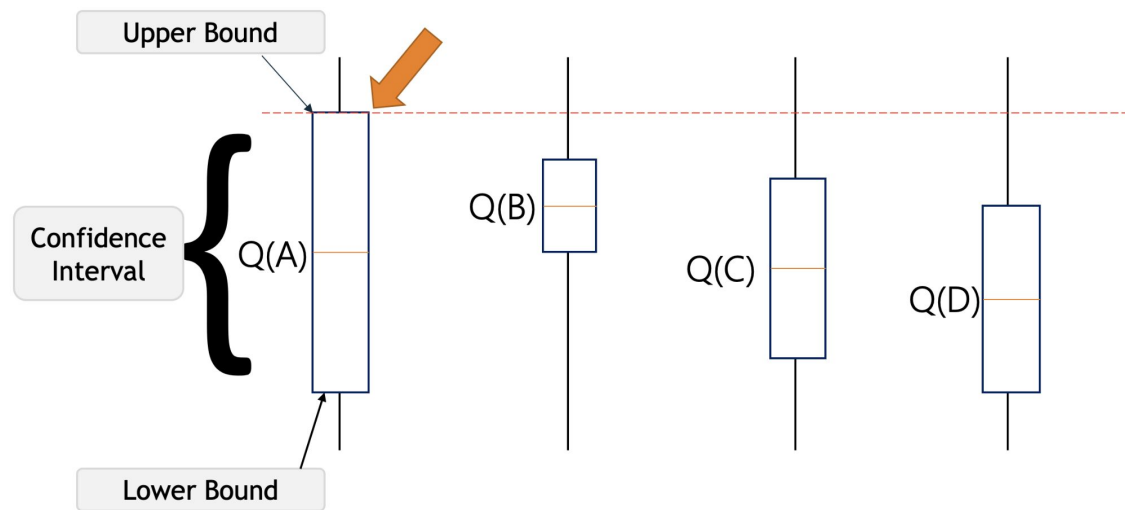
1. Random perturbation (ϵ -greedy)
2. Optimistic initialization (oi)
3. **Uncertainty-based** (**ucb**)

UCB

'If we could track the remaining uncertainty about each arm, we could more adaptively switch between exploration and exploitation'

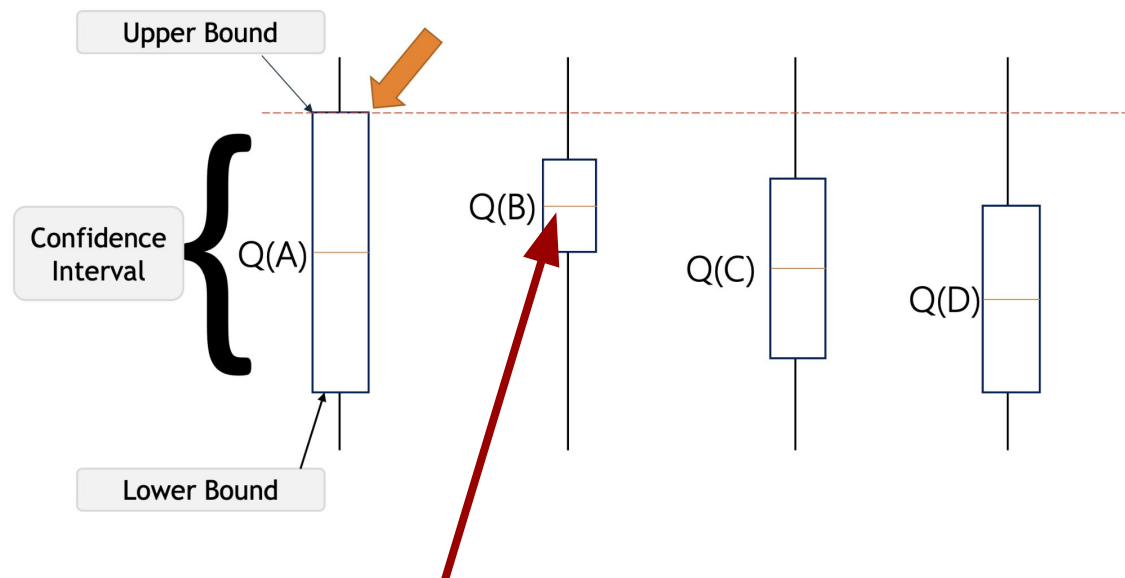
UCB

'If we could track the remaining uncertainty about each arm, we could more adaptively switch between exploration and exploitation'



UCB

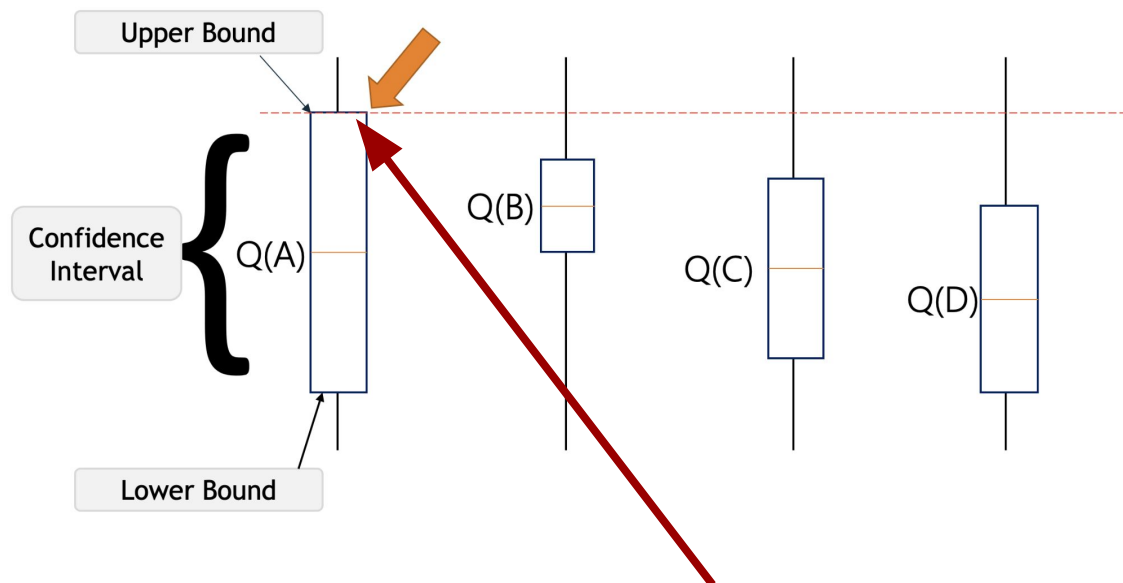
'If we could track the remaining uncertainty about each arm, we could more adaptively switch between exploration and exploitation'



Only considering the mean: action B seems most promising...

UCB

'If we could track the remaining uncertainty about each arm, we could more adaptively switch between exploration and exploitation'



When we incorporate the uncertainty: action A may have more potential!

UCB

'Optimism in the face of uncertainty'

UCB

'Optimism in the face of uncertainty'

- Estimate the mean and standard deviation of each action.
- Select action with *highest upper confidence bound* (UCB)

UCB

'Optimism in the face of uncertainty'

- Estimate the mean and standard deviation of each action.
- Select action with *highest upper confidence bound* (UCB)

$$a_{\text{UCB}} = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right]$$

UCB

'Optimism in the face of uncertainty'

- Estimate the mean and standard deviation of each action.
- Select action with *highest upper confidence bound* (UCB)

$$a_{\text{UCB}} = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right]$$

Mean

UCB

'Optimism in the face of uncertainty'

- Estimate the mean and standard deviation of each action.
- Select action with *highest upper confidence bound* (UCB)

$$a_{\text{UCB}} = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right]$$

c = exploration parameter

UCB

'Optimism in the face of uncertainty'

- Estimate the mean and standard deviation of each action.
- Select action with *highest upper confidence bound* (UCB)

$$a_{\text{UCB}} = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right]$$

Approximately the standard
error of mean
(decreases as square root of
number of visits to action)

UCB

'Optimism in the face of uncertainty'

- Estimate the mean and standard deviation of each action.
- Select action with *highest upper confidence bound (UCB)*

$$a_{\text{UCB}} = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right]$$

Important: when an action is untried ($n(a)=0$), we treat it's UCB estimate as ∞

UCB

'Optimism in the face of uncertainty'

- Estimate the mean and standard deviation of each action.
- Select action with *highest upper confidence bound (UCB)*

$$a_{\text{UCB}} = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right]$$


Important: when an action is untried ($n(a)=0$), we treat it's UCB estimate as ∞

Effect: always prefer untried action over action that has been sampled

UCB pseudocode

Algorithm 4: UCB bandit algorithm.

Input: Exploration parameter $c \in \mathbb{R}^+$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = 0$, $n(a) = 0 \forall a \in \mathcal{A}$

for $t = 1..T$ **do**
$$a_t = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right] \quad /* \text{ UCB action } */$$
$$r_t \sim p(r|a_t) \quad /* \text{ Sample reward } */$$
$$n(a_t) \leftarrow n(a_t) + 1 \quad /* \text{Update count} */$$
$$Q(a_t) \leftarrow Q(a_t) + \frac{1}{n(a_t)} [r_t - Q(a_t)] \quad /* \text{Incr. update mean} */$$

end

UCB pseudocode

Algorithm 4: UCB bandit algorithm.

Input: Exploration parameter $c \in \mathbb{R}^+$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = 0$, $n(a) = 0 \forall a \in \mathcal{A}$

for $t = 1 \dots T$ **do**

$a_t = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right]$ */* UCB action */*

$r_t \sim p(r|a_t)$ */* Sample reward */*

$n(a_t) \leftarrow n(a_t) + 1$ */* Update count */*

$Q(a_t) \leftarrow Q(a_t) + \frac{1}{n(a_t)} [r_t - Q(a_t)]$ */* Incr. update mean */*

end

Needs exploration parameter c as input (higher c = more exploration)

UCB pseudocode

Algorithm 4: UCB bandit algorithm.

Input: Exploration parameter $c \in \mathbb{R}^+$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = 0, n(a) = 0 \forall a \in \mathcal{A}$

```
for  $t = 1..T$  do
```

$$a_t = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right] \quad /* \text{ UCB action } */$$
 $r_t \sim p(r|a_t)$ /* Sample reward */
$$n(a_t) \leftarrow n(a_t) + 1 \quad /* \text{Update count} */$$
$$Q(a_t) \leftarrow Q(a_t) + \frac{1}{n(a_t)} [r_t - Q(a_t)] \quad /* \text{Incr. update mean} */$$

end

Initialize means and counts to 0

UCB pseudocode

Algorithm 4: UCB bandit algorithm.

Input: Exploration parameter $c \in \mathbb{R}^+$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = 0$, $n(a) = 0 \forall a \in \mathcal{A}$

for $t = 1 \dots T$ **do**

$$a_t = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right]$$

/ UCB action */*

$$r_t \sim p(r|a_t)$$

/ Sample reward */*

$$n(a_t) \leftarrow n(a_t) + 1$$

/ Update count */*

$$Q(a_t) \leftarrow Q(a_t) + \frac{1}{n(a_t)} [r_t - Q(a_t)]$$

/ Incr. update mean */*

end

Use UCB formula for action selection

UCB pseudocode

Algorithm 4: UCB bandit algorithm.

Input: Exploration parameter $c \in \mathbb{R}^+$, maximum number of timesteps T .

Initialization: Initialize $Q(a) = 0$, $n(a) = 0 \forall a \in \mathcal{A}$

for $t = 1..T$ **do**
$$a_t = \arg \max_a \left[Q(a) + c \cdot \sqrt{\frac{\ln t}{n(a)}} \right] \quad /* \text{ UCB action } */$$
 $r_t \sim p(r|a_t)$ /* Sample reward */
$$n(a_t) \leftarrow n(a_t) + 1 \quad /* \text{Update count} */$$
$$Q(a_t) \leftarrow Q(a_t) + \frac{1}{n(a_t)} [r_t - Q(a_t)] \quad \text{/* Incr. update mean */}$$

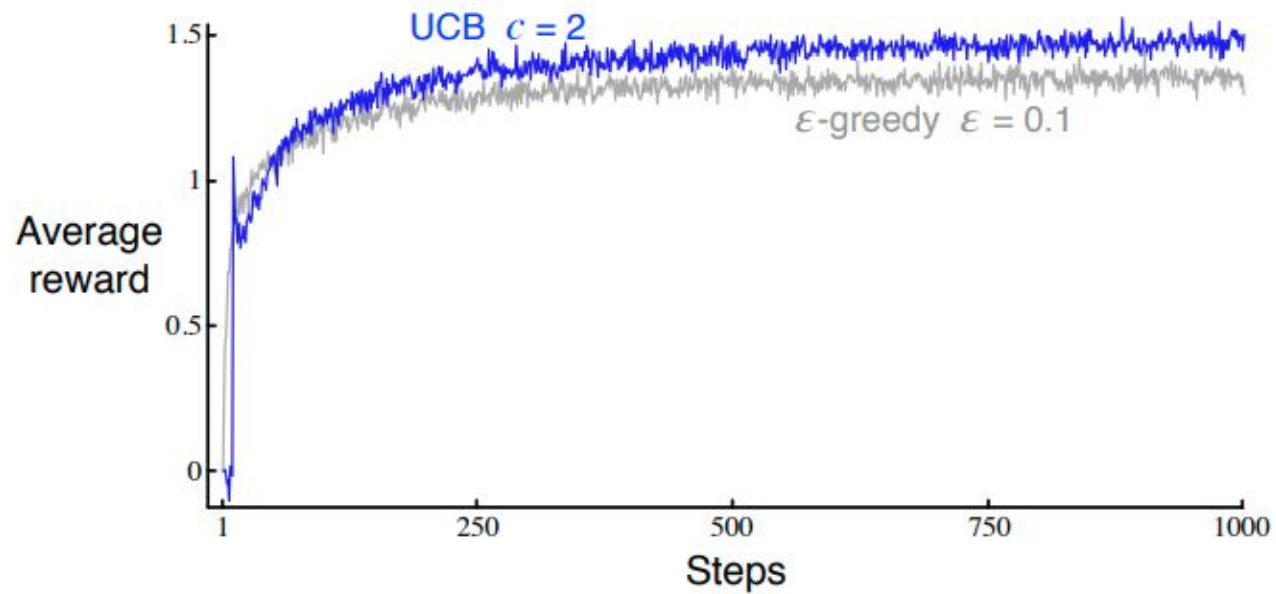
end

Incremental update of the mean

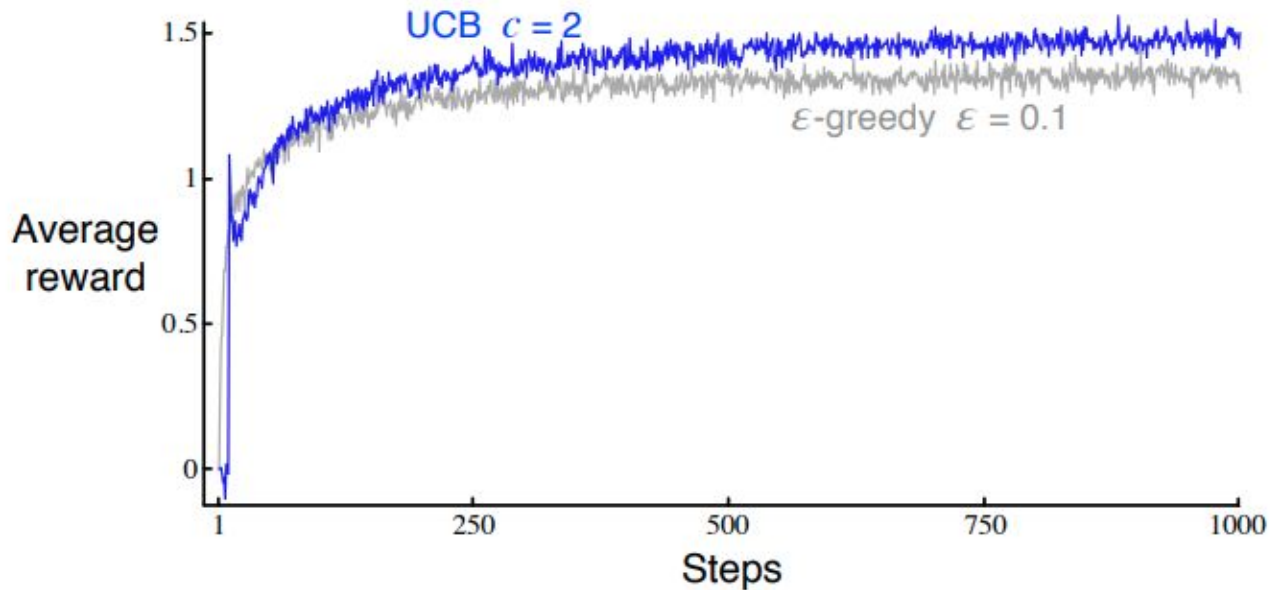
UCB performance



UCB performance



UCB performance



Main benefit of UCB over other approaches:

Adaptively transitions (in a data-dependent way) from exploration (little data, high uncertainty) to exploitation (much data, small uncertainty)

Exploration hyperparameters



Exploration hyperparameters

All algorithms typically have (at least) one parameter that tunes the exploration/exploitation trade-off

Exploration hyperparameters

All algorithms typically have (at least) one parameter that tunes the exploration/exploitation trade-off

- e-greedy: **epsilon**
- optimistic initialization: **initial value**
- upper confidence bound: **c**

Exploration hyperparameters

All algorithms typically have (at least) one parameter that tunes the exploration/exploitation trade-off

- e-greedy: **epsilon**
- optimistic initialization: **initial value**
- upper confidence bound: **c**

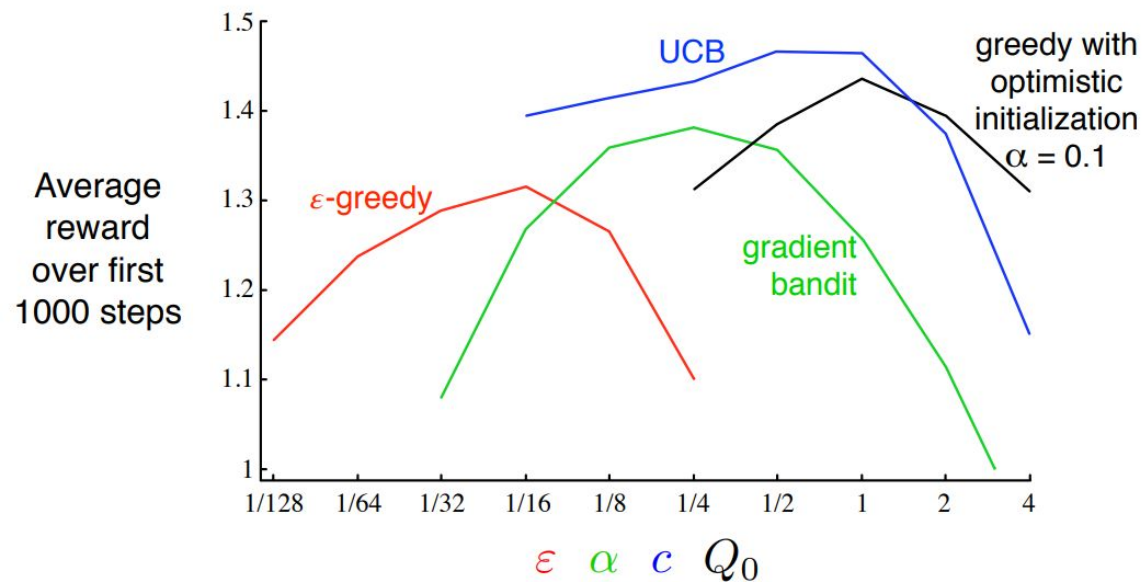
1. For the examples above: a higher setting gives more exploration

Exploration hyperparameters

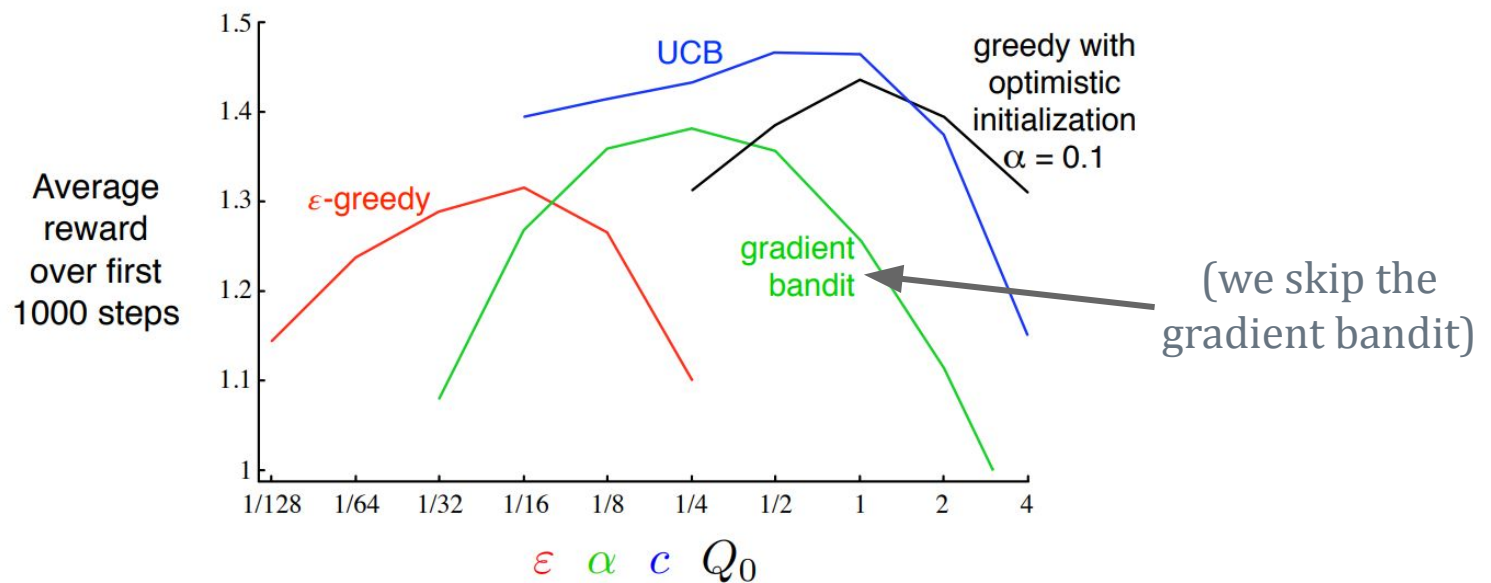
All algorithms typically have (at least) one parameter that tunes the exploration/exploitation trade-off

- e-greedy: **epsilon**
 - optimistic initialization: **initial value**
 - upper confidence bound: **c**
-
1. For the examples above: a higher setting gives more exploration
 2. You typically need to tune this parameter per problem

Exploration-exploitation trade-off

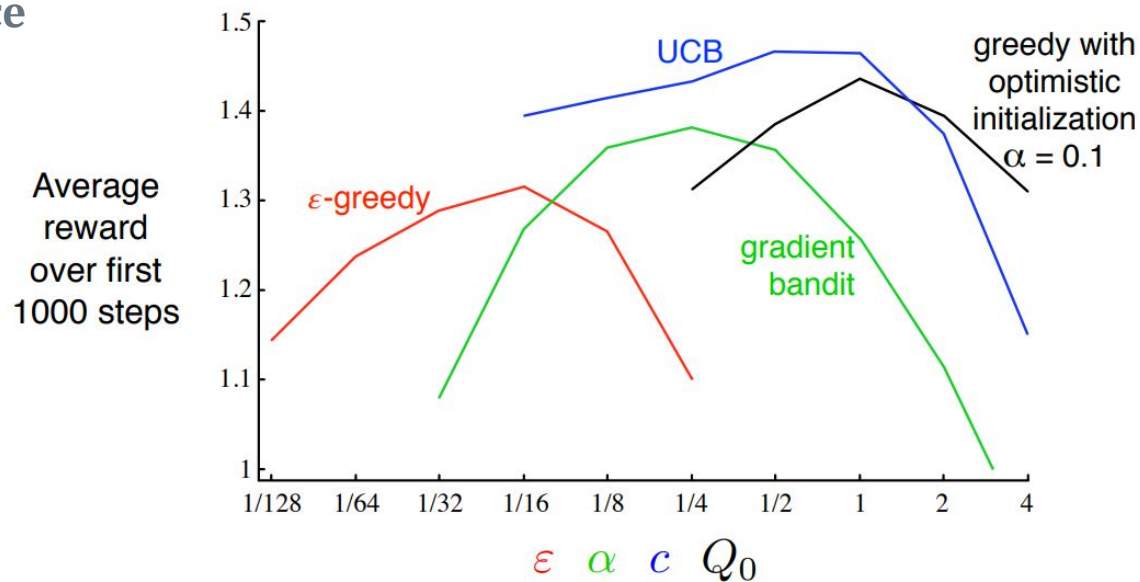


Exploration-exploitation trade-off



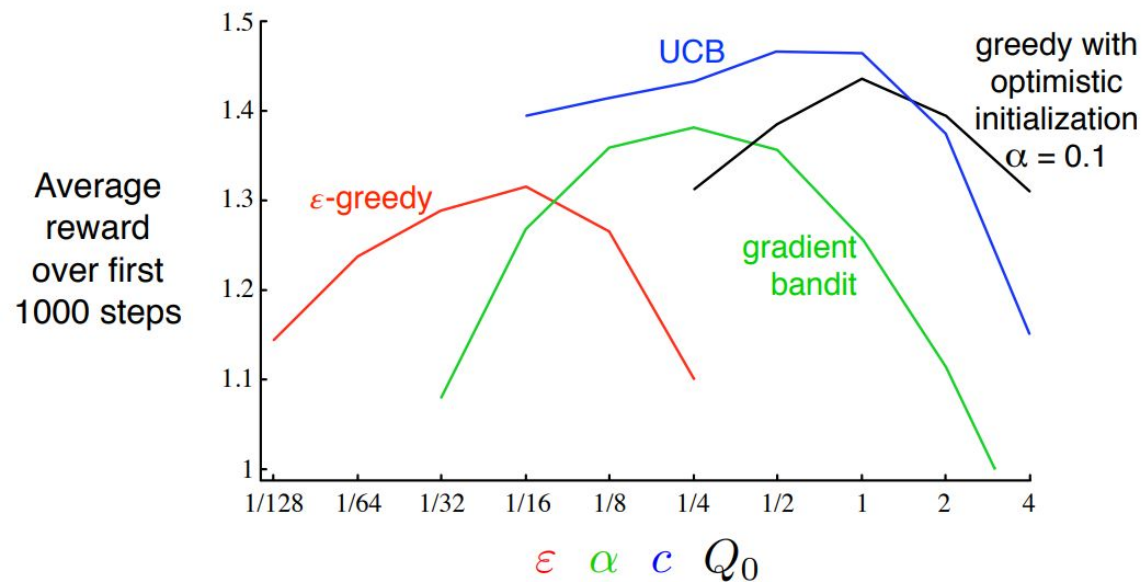
Exploration-exploitation trade-off

algorithm
performance



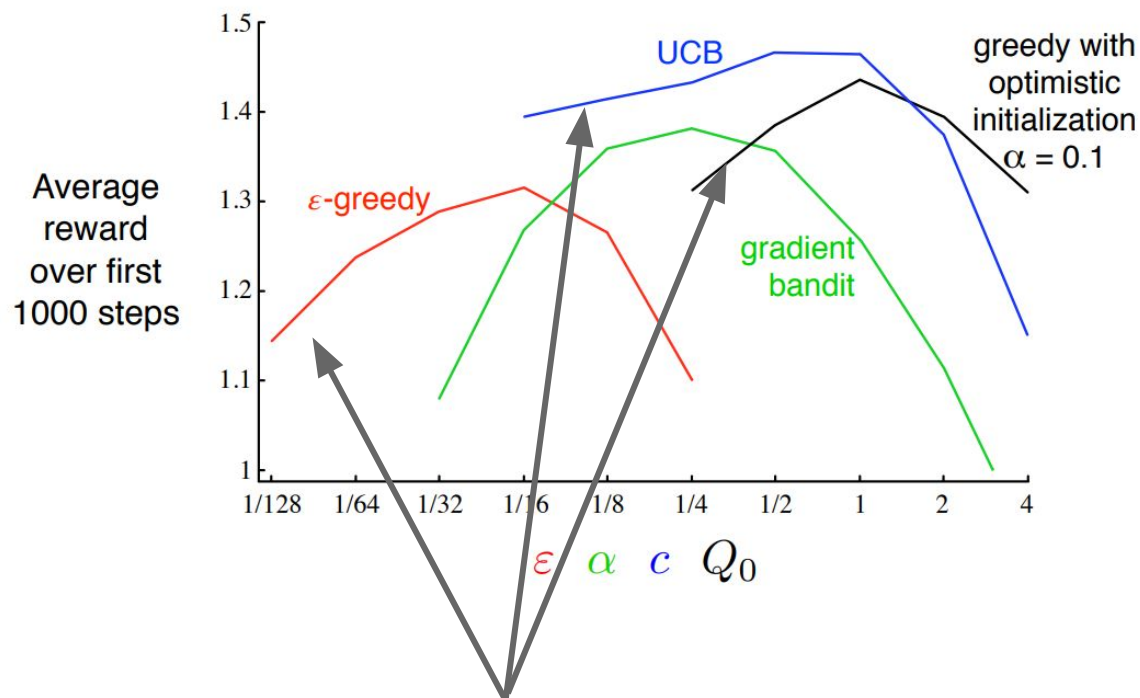
exploration
hyperparameter

Exploration-exploitation trade-off



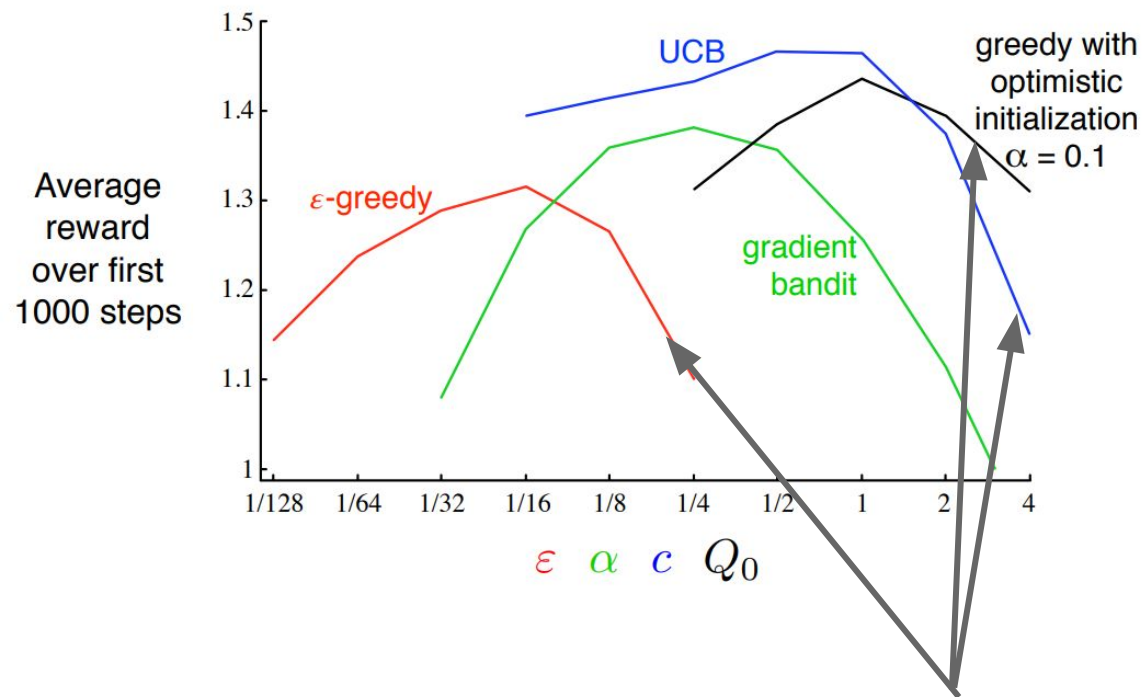
You don't want too little exploration nor too much exploration

Exploration-exploitation trade-off



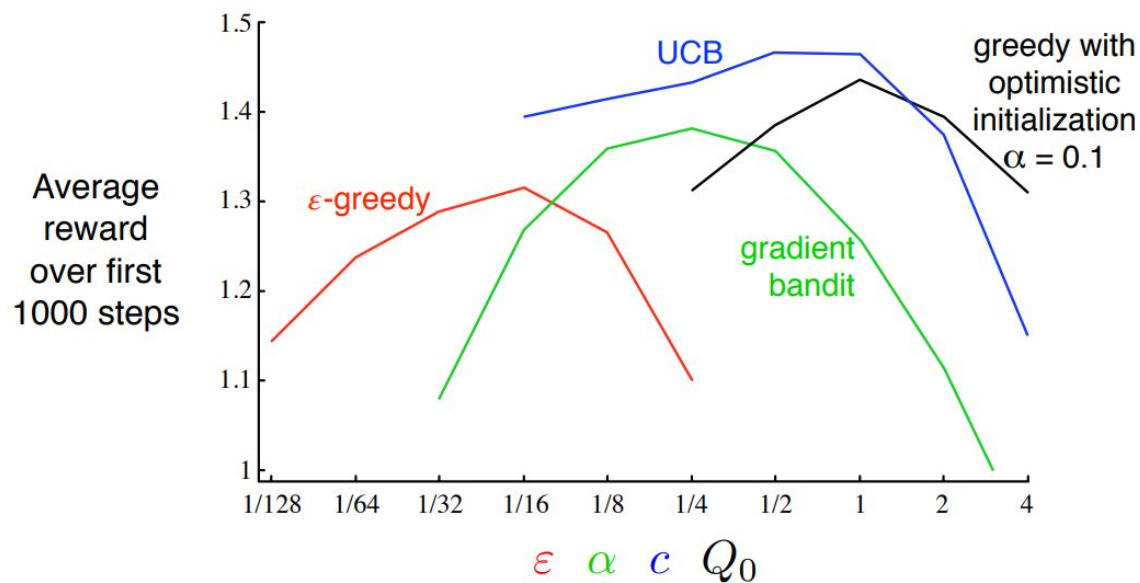
You don't want too little exploration nor too much exploration

Exploration-exploitation trade-off



You don't want too little exploration nor too much exploration

Exploration-exploitation trade-off

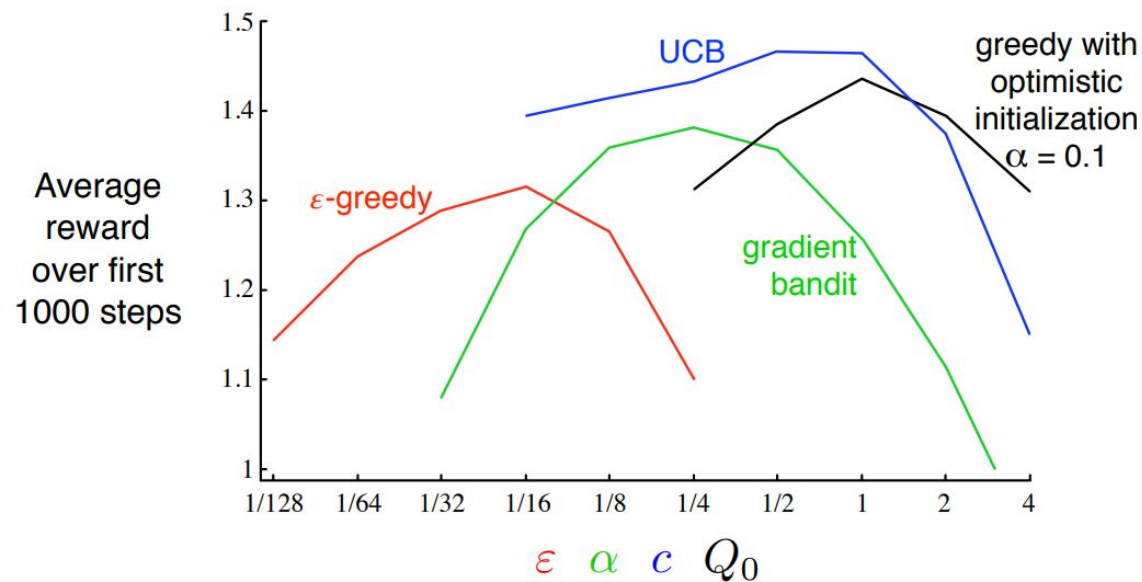


You don't want too little exploration nor too much exploration

=

trade-off

Exploration-exploitation trade-off



You don't want too little exploration nor too much exploration

=

trade-off (as with most hyperparameters)

Advanced bandit topics

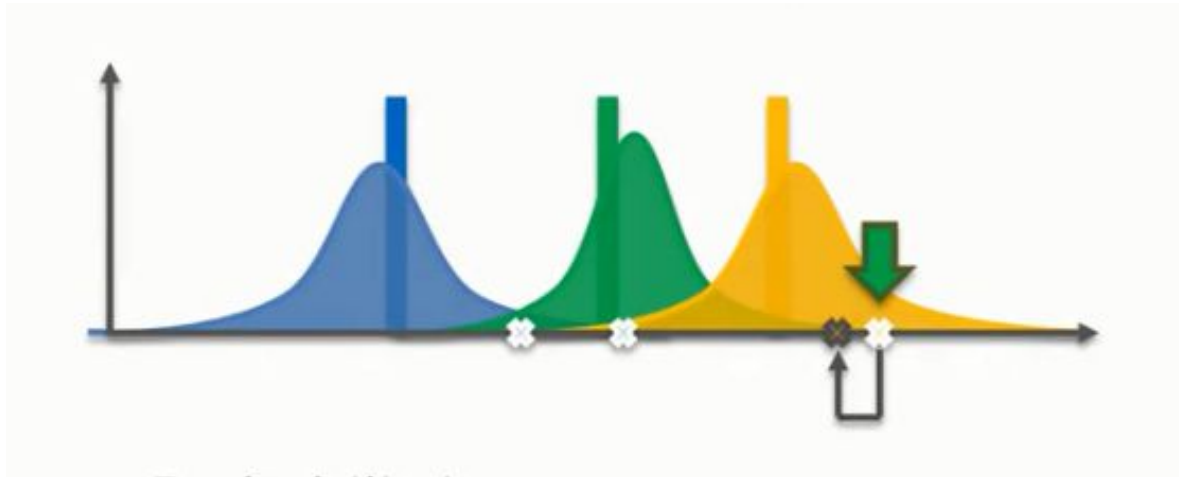
- Much research, many more exploration strategies.
 - e.g. Bayesian methods:

Advanced bandit topics

- Much research, many more exploration strategies.
 - e.g. Bayesian methods:
 1. Estimate a posterior distribution for each mean estimate

Advanced bandit topics

- Much research, many more exploration strategies.
 - e.g. Bayesian methods:
 1. Estimate a posterior distribution for each mean estimate
 2. Use Thompson sampling (probability matching):
select action according to probability that it is the best one



Advanced bandit topics

- Much research, many more exploration strategies.
 - e.g. Bayesian methods:
 1. Estimate a posterior distribution for each mean estimate
 2. Use Thompson sampling (probability matching):
select action according to probability that it is the best one



Very theoretical branch of research (more than RL):
proving convergence properties of algorithms.

Part 5:

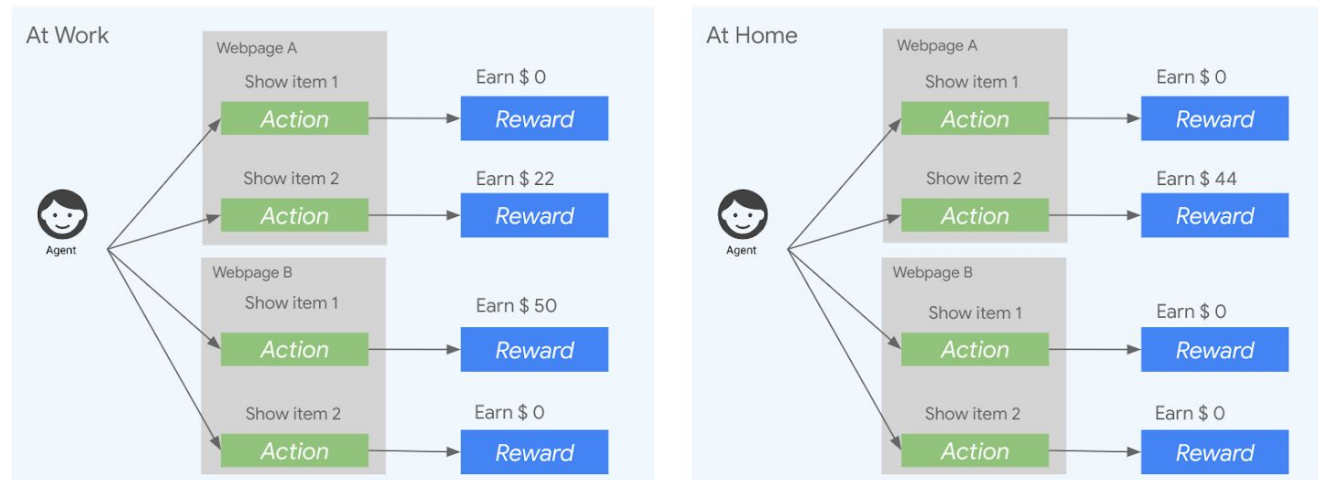
Contextual bandits & MDPs

Contextual bandit

Often the reward distribution of the bandit you face depends on *context*

Contextual bandit

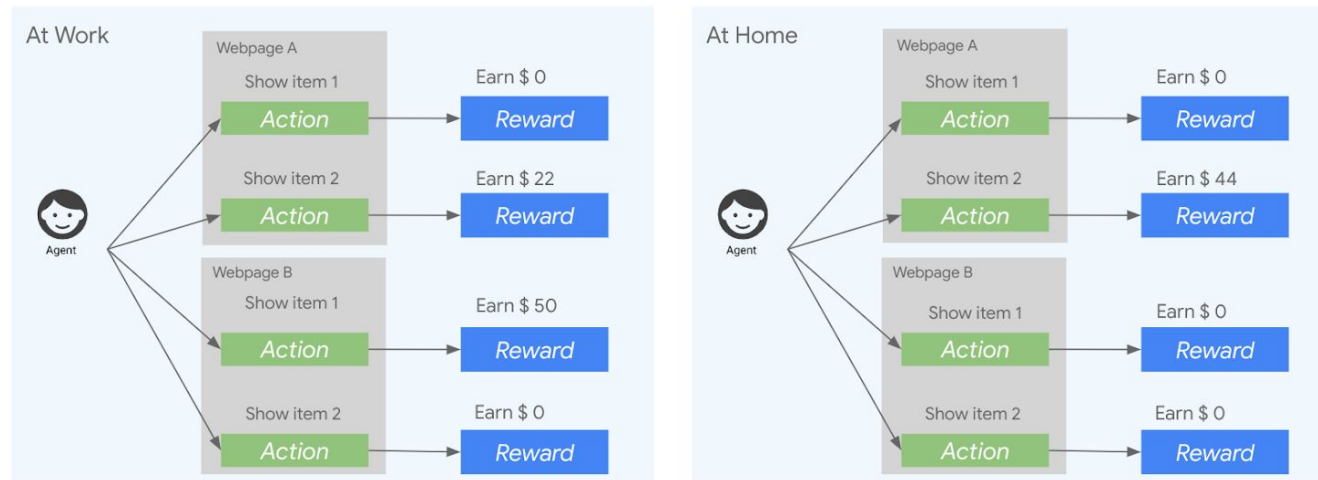
Often the reward distribution of the bandit you face depends on *context*



Reward is conditional to the state of the environment:

Contextual bandit

Often the reward distribution of the bandit you face depends on *context*



Reward is conditional to the state of the environment:

Contextual bandit: reward distribution depends on context state s (such as whether the person views a webpage at *home* or at *work*)

Markov Decision Process

When the state also changes based on our action we call it a
Markov Decision Process (MDP)

Markov Decision Process

When the state also changes based on our action we call it a
Markov Decision Process (MDP)



Bandit

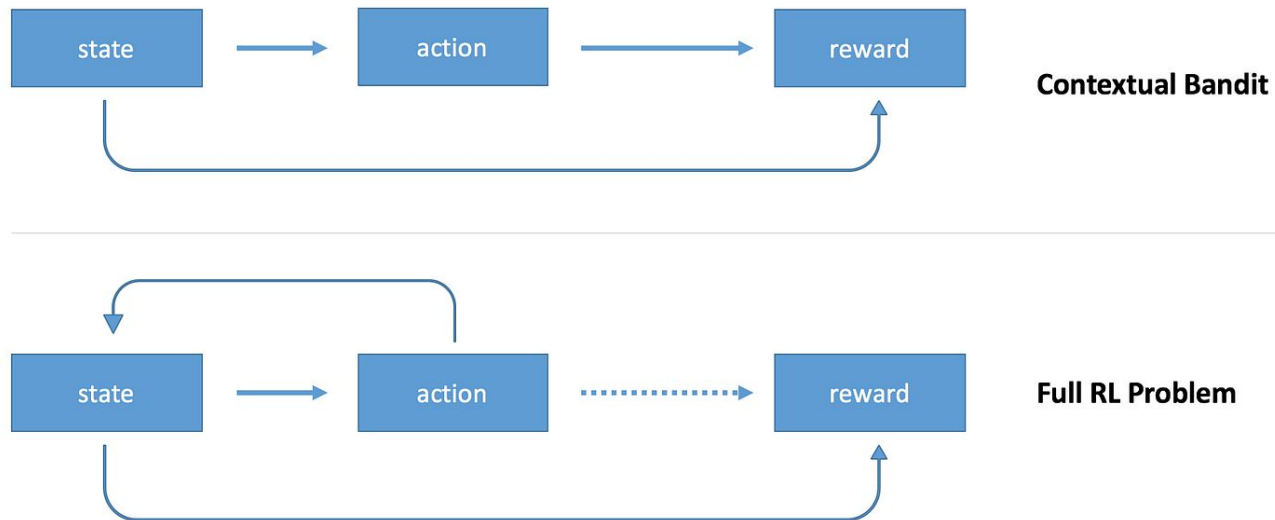
Markov Decision Process

When the state also changes based on our action we call it a
Markov Decision Process (MDP)



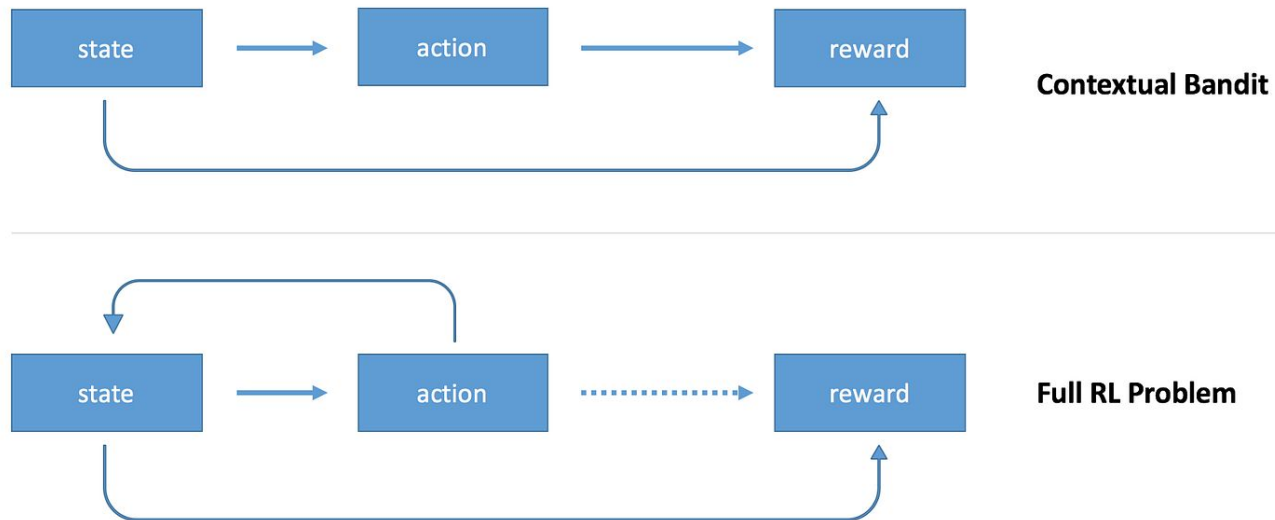
Markov Decision Process

When the state also changes based on our action we call it a
Markov Decision Process (MDP)



Markov Decision Process

When the state also changes based on our action we call it a
Markov Decision Process (MDP)



Next week!

To Do

Read:

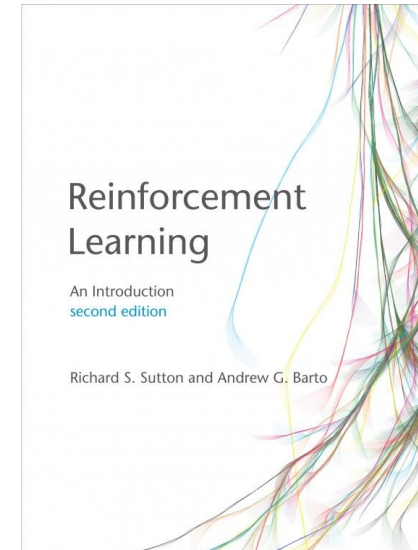
Assignment:

To Do

Read:

1. Sutton & Barto, Chapter 2 (multi-armed bandit)
2. Lecture slides and lecture notes

Assignment:



Free online version:
[http://incompleteideas.net
/book/RLbook2020.pdf](http://incompleteideas.net/book/RLbook2020.pdf)

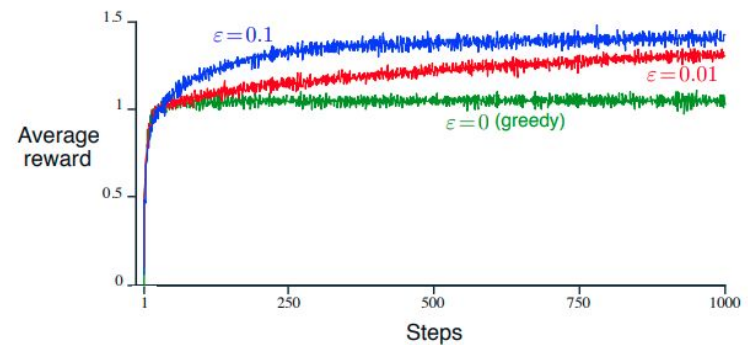
To Do

Read:

1. Sutton & Barto, Chapter 2 (multi-armed bandit)
2. Lecture slides and lecture notes

Assignment:

1. Work on Assignment 1a
(about Bandits)



Questions?